

The Australian National University
2600 ACT | Canberra | Australia



Australian
National
University

School of Computing

College of Engineering, Computing
and Cybernetics (CECC)

Sprinkle Magic on the Dance: Enriching a Verified Choreographic Language with a Simply Typed Lambda Calculus

— Honours project (S1/S2 2024)

A thesis submitted for the degree
Bachelor of Advanced Computing (Research and Development)

By:
Xin Lu

Supervisor:
Dr. Michael Norrish

February 2025

Declaration:

I declare that this work:

- upholds the principles of academic integrity, as defined in the [University Academic Misconduct Rules](#);
- is original, except where collaboration (for example group work) has been authorised in writing by the course convener in the class summary and/or Wattle site;
- is produced for the purposes of this assessment task and has not been submitted for assessment in any other context, except where authorised in writing by the course convener;
- gives appropriate acknowledgement of the ideas, scholarship and intellectual property of others insofar as these have been used;
- in no part involves copying, cheating, collusion, fabrication, plagiarism or recycling.

February, Xin Lu

Acknowledgements

I would like to express my deepest gratitude to Michael Norrish, my thesis advisor, for his continuous support, guidance, and encouragement throughout the research and writing of this thesis. His insights and expertise were invaluable in shaping this work. It was also incredibly fun to develop the ideas for this project with him.

I am deeply grateful to my partner, Yuchen Li, for his unwavering support and delicious dishes throughout this project.

A special thank you goes to my friends, Selene Kim and Amelia Genova, for always being there to listen and offer words of encouragement. Your friendship has been a source of strength throughout this academic journey.

I would also like to acknowledge Peter Hoefner and Fabian Muehlboeck from COMP3610 for being the most inspiring lecturers on programming language design. Additionally, I want to extend my gratitude to Weiyu Wang for being a passionate tutor throughout the course.

I am thankful to my colleagues from the COMP1110 tutor team for bringing laughter to every meeting and assisting me with scheduling during the busiest times at work.

Finally, I owe my deepest appreciation to my parents, Huaqin Liao and Chao Lu, for their endless love, support, and encouragement. They have been my greatest source of inspiration, and their belief in my abilities has carried me through this journey.

Xin Lu
25-Oct-2024

Abstract

Distributed systems are ubiquitous but writing endpoint programs can be error-prone since mismatched message sending and receiving can lead to errors such as deadlock, where the system indefinitely awaits a message. Choreography offers a solution by providing a global description of how messages are exchanged among endpoints, where message mismatches are disallowed — a property called “deadlock-free by design”. The global choreography is then projected into process models for each endpoint, preserving the deadlock-free property.

While many choreography languages focus on message exchange behaviors, few address the local computations occurring within endpoints. Most current languages assume local computation results or delegate them to external languages. While this offers a reasoning ground for studying message exchange behaviours of choreography, when it comes to writing a concrete choreography program, the former can only exchange literal values and the latter leads to cumbersome code due to the addition of an external computation program which typically involves conversions between choreography values and external data types.

Hence in this thesis, we extend Kalas, a state-of-the-art choreography language with verified end-to-end compilation, with a local language **Sprinkles**, such that local computations are handled gracefully within a few lines of codes. Moreover, it also allows us to formally analyse the message exchange behaviours of choreography when local computations are considered. Our proofs show that the strong normalisation property of **Sprinkles** implies *progress* for the enriched Kalas, and that the combination of type soundness and strong normalisation of **Sprinkles** implies type preservation for non-recursive, synchronous transitions in the enriched Kalas.

Table of Contents

1	Introduction	1
2	Background	5
2.1	Choreographic Programming	5
2.2	Interactive Theorem Proving	6
2.3	Kalas	6
3	Related Work	9
3.1	Choreography Models	9
3.1.1	Typing System for Choreography	9
3.1.2	Handling Exceptions	10
3.2	Approach to Strong Normalisation	11
4	Sprinkles: A Simply Typed Lambda Calculus	13
4.1	Syntax	13
4.2	Statics	14
4.2.1	Syntax and Typing Rules	15
4.2.2	Typing Examples	16
4.3	Dynamics	17
4.3.1	Function Closure	18
4.3.2	Main Properties	19
4.4	Type Soundness	20
4.4.1	Lemmas	20
4.4.2	Type Soundness	24
5	Strong Normalisation	27
5.1	Formalisation	27
5.2	Auxillary Lemmas	29
5.3	The Fundamental Lemma	31
5.3.1	Strategy	31
5.3.2	Proof	32
5.3.3	Strong Normalisation	33
6	Kalas Enriched with Sprinkles	35
6.1	Syntax	35
6.2	Statics	36

6.2.1	Syntax and Typing Rules	36
6.2.2	Typing Examples	37
6.3	Dynamics	38
6.4	Progress	38
6.4.1	Formalisation	39
6.4.2	Lemmas	39
6.4.3	Proof	41
6.5	Type Preservation	41
6.6	Discussions on Kalas Properties	42
6.6.1	Transitions in Larger States	42
6.6.2	Synchronous Transitions	43
7	Concluding Remarks	47
7.1	Conclusion	47
7.2	Future Work	47
A	Appendix	49

Introduction

Distributed systems consist of multiple endpoints that communicate by exchanging messages, operating asynchronously and in parallel as these messages are sent and received among the various endpoints. However, programming distributed systems is notoriously error-prone because programmers must implement the communication protocol by developing individual endpoint programs. Mismatched message sending and receiving can lead to errors such as *deadlock*, where the system indefinitely waits for a message.

Choreographic programming arises as a formalisation to address this issue by providing a concrete global description of how the messages are exchanged between endpoints in a distributed system. A choreography program is written in a similar style to the "Alice and Bob" notation by [Needham and Schroeder \(1978\)](#):

1. Alice $\rightarrow$ Bob : *key*
2. Bob $\rightarrow$ Alice : *message*

Thus message mismatches are disallowed from the choreographic perspective. This property is known as *deadlock-free-by-design*. The global choreography can then be projected into process models for each endpoint via EndPoint Projection (EPP), with properties such as deadlock free by design preserved ([Hallal et al., 2018](#)).

When analysing message exchange behaviors in choreography, such as multiparty sessions, asynchrony, and parallelism, it is reasonable to assume good behaviors in local computation ([Montesi and Yoshida, 2013](#); [Cruz-Filipe and Montesi, 2017](#); [Carbone and Montesi, 2013](#)). Consequently, most choreography languages focus primarily on message exchange behaviours, with few paying attention to the local computation occurring in individual endpoints. Although [Hirsch and Garg \(2022\)](#) demonstrates that for their assumed local language, as long as it exhibits type preservation and progress, the choreography inherits these properties, this result may not hold true for all local languages.

Additionally, when it comes to writing the choreography program that implements concrete system behaviours, if local computation is ever required by the system, one must describe the inputs and outputs of local computations. Current work either delegate this part to an assumed well-behaved external implementation, for example, Kalas [Pohjola et al. \(2022\)](#) and Pirouette [Hirsch and Garg \(2022\)](#), or provide a basic framework where only Church numerals are considered, as

in Core Choreography (CC) [Cruz-Filipe and Montesi \(2017\)](#). This makes writing choreography program with local computation implementation an unpleasant experience. For instance, a choreography program in Kalas, where the client computes modulo locally using input from the server and then sends the result back, will look like:

Example 1.0.1 (Local computation — Modulo).

Kalas	External computation
<pre> 1. server.var → client.x; 2. let v@client = mod(x) in 3. client.v → server.result;</pre>	<pre> fun mod x = case Option.map (fn s ⇒ valOf (Int.fromString s)) (hd x) of None ⇒ None Some n ⇒ Some [Int.toString (n MOD y)]</pre>

In Kalas, processes communicate exclusively via strings. Therefore, in the external implementation of `mod(x)`, the string value of x must first be converted to a number. Then client computes the modulo on the converted input. Afterward, the result is converted back to a string before updating the client process variable v . We can easily see from this example that how data type conversions between choreography and external language lead to cumbersome code.

Thus this thesis extends Kalas, a verified choreography language with machine-checked end-to-end compilation, by introducing **Sprinkles**, a simple language of expressions over types such as integers, strings and booleans. Using the extended Kalas, local computations can be handled gracefully within a few lines of code. The previous choreography where client computes the modulo of input integer can be written as in [Example 1.0.2](#).

By providing a concrete local language syntax and semantics, we are able to formally analyse the message exchange behaviours of choreography in terms of progress and type preservation when local computations are considered. We show that type soundness and strong normalisation properties of **Sprinkles** lead to progress for the enriched Kalas. Our semantics and typing system also allow us to show type preservation for non-recursive and synchronous transitions in the enriched Kalas.

Example 1.0.2 (Local computation with Sprinkles — Modulo).

```

1. server.var → client.x;
2. let v@client = StrOf ((NumOf (Var x)) Mod (Var y)) in
3. client.v → server.result;
```

To summarise, this thesis provides three main contributions.

- The first contribution is **Sprinkles**, a simply typed lambda calculus with rich types. Following the approach taken by [Owens et al. \(2016\)](#), we use a functional big-step semantics with environments and closures to ensure the evaluation function for **Sprinkles**' expressions is total. Our evaluation strategy is call-by-value. We also provide a typing system and give type soundness proof for the proposed semantics and typing rules. **Sprinkles** is implemented using the higher-order logic proof assistant HOL4 ([Slind and Norrish, 2008](#)) and all the proofs are conducted within HOL4.
- The second contribution is the strong normalisation proof for **Sprinkles**. We follow the standard practice for proving strong normalisation in simply typed lambda calculus, with a specific case for function closure in the definition of the strong normalisation relation.

- The third contribution is Kalas enriched with **Sprinkles**. Besides common data types such as integer, string, and boolean, we also add function, pair, and sum types to the local computation in Kalas. Common operators for our data types are included, such as addition, modulo, and negation. Integer-string conversion is implemented as well for the integration with Kalas. Last but not least, we prove the enriched Kalas enjoys an important property: *progress*. We also show type preservation holds for non-recursive, synchronous transitions in the enriched Kalas.

Background

2.1 Choreographic Programming

A distributed system is defined as a collection of autonomous computing elements that operate according to users' expectations (Mullender, 1990). Message exchanges are at the core of distributed system design, as they enable nodes to collaborate and share resources. Without such exchanges, there would be no need to connect different nodes within a single network. In a traditional approach to describe a distributed system, one must provide a detailed specification of the operations at each individual node. For example, communication between node A and node B involves A sending a message and B receiving it. However, this approach can lead to issues such as mismatched message sending and receiving, potentially resulting in deadlocks or race conditions.

Choreography emerges as an effort to eliminate message mismatches in distributed systems by offering a global description of how messages should be exchanged. This method is similar to dance choreography, where the focus is on the overall coordination of steps and movements for the entire performance, rather than detailing the control points of individual dancers.

While it is exciting to eliminate mismatched messages from the choreography perspective, choreography cannot be executed directly on individual nodes within a system. As a result, implementing endpoint programs is still necessary. This led to the idea of EndPoint Projection (EPP), which involves projecting a choreography into endpoint programs so that each endpoint correctly implements the behaviours defined by its role in the choreography. This concept was first outlined in the design document for the Web Services Choreography Description Language (WS-CDL) (W3C WS-CDL Working Group, 2005), and was subsequently formalised into the theory of EPP by Carbone et al. (2007). In this context, two key properties are established: *soundness* and *completeness*. Soundness ensures that all projected endpoint communications conform to the choreography description, while completeness guarantees that all communications specified in the choreography are reflected in the projected endpoint codes.

2.2 Interactive Theorem Proving

Higher Order Logic (HOL) is a programming language developed on top of the metalanguage ML (Gordon, 1988). ML is an interactive programming language that allows users to evaluate expressions, with the corresponding values and types being printed. Users can also perform declarations to bind values to names. In HOL, terms are represented as an ML type called *term*, while the types of HOL terms are represented as an ML type called *type*.

We are mainly interested in using HOL as a proof assistant. HOL supports proofs by natural deduction. In this context, a sequent in HOL is defined as a pair (Γ, t) , where Γ represents a set of assumptions and t denotes the conclusion formula. A theorem in HOL is a sequent that is either an axiom or a sequent derived from other theorems using rules of inference.

A key feature of HOL as a proof assistant is its support for *goal-oriented* proving through the use of tactics. In a goal-oriented proof, the goal is broken down into subgoals through justifications. This process continues until the subgoals become straightforward to prove. A *tactic* is an ML function that reduces the current goal into subgoals. For example, the conjunction tactic takes a goal of the form $p \wedge q$ and breaks it into two subgoals: p and q .

HOL includes many built-in libraries that cover useful theories for option types, natural numbers, set operations, and more. These theories can be loaded into the current working file, allowing users to build definitions and proofs based on the imported theories. Consequently, we develop **Sprinkles** and its related theorems by first creating HOL definitions for our syntax, functions, and relations based on the imported theories. We then apply HOL tactics to prove the stated theorems in a goal-oriented manner. The most frequently used theory for this project, aside from common theories like natural numbers and set operations, is the finite map theory. Both Kalas' states and **Sprinkles**'s environments are implemented as finite maps. This project is also based on HOL4, the latest supported version of HOL (Slind and Norrish, 2008).

2.3 Kalas

Kalas is a state-of-the-art choreography language featuring verified end-to-end compilation. Its syntax is outlined in Definition 1 and is similar to that of Core Choreography (CC) (Cruz-Filipe and Montesi, 2017). Kalas supports recursions and sequencing. Besides the synchronous transitions, Kalas' transition rules also handle parallel and asynchronous transitions (Definition 2). Most importantly, a compiler with an end-to-end, machine-checked proof of correctness for Kalas is provided.

The compilation process for Kalas consists of several phases, during which the top-level choreography is projected into endpoint process algebra. The process programs are further compiled phase by phase to address message sizes and the Kalas fixpoint operator. The compiler reaches CakeML (Kumar et al., 2014), a functional programming language with verified compilation, in the final compilation phase, which gives Kalas semantics preservation down to the machine code. Soundness and completeness proofs are provided for each compilation phase to ensure operational correctness at every stage. As a result, Kalas enjoys network-level deadlock free and correctness.

Kalas is implemented using HOL4, with all its proofs conducted through the HOL proof assistant. This foundation leads us to implement **Sprinkles** and its associated proofs using HOL.

Definition 1 (Kalas syntax). *Choreographies in Kalas, ranged over by C , are inductively defined by the grammar*

$$\begin{array}{llll}
C ::= & p_1.v_1 \rightarrow p_2.v_2; C & (com) & p_1 \rightarrow p_2[b]; C & (sel) \\
& \mathbf{if} \ v@p \ \mathbf{then} \ C_1 \ \mathbf{else} \ C_2 & (if) & \mathbf{let} \ v@p = f(\tilde{v}) \ \mathbf{in} \ C & (let) \\
& \mu X. C & (fix) & X & (var) \\
& \mathbf{0} & (nil) & &
\end{array}$$

Definition 2 (Parallel and asynchronous transitions in Kalas).

$$\begin{array}{c}
\text{COM-S} \frac{s \triangleright C \xrightarrow[l]{\alpha} s' \triangleright C' \quad p_1 \notin \text{fp}(\alpha) \quad p_2 \notin \text{fp}(\alpha)}{s \triangleright p_1.v_1 \rightarrow p_2.v_2; C \xrightarrow[l]{\alpha} s' \triangleright p_1.v_1 \rightarrow p_2.v_2; C'} \\
\text{COM-A} \frac{s \triangleright C \xrightarrow[l]{\alpha} s' \triangleright C' \quad p_1 \in \text{fp}(\alpha) \quad \text{wv}(\alpha) \neq (v_1, p_1) \quad p_2 \notin \text{fp}(\alpha)}{s \triangleright p_1.v_1 \rightarrow p_2.v_2; C \xrightarrow[(p_1.v_1 \triangleright p_2.v_2)::l]{\alpha} s' \triangleright p_1.v_1 \rightarrow p_2.v_2; C'}
\end{array}$$

Related Work

3.1 Choreography Models

3.1.1 Typing System for Choreography

Choreography models can ensure deadlock freedom without relying on a typing system. This property is typically proven through structural induction on the choreography's semantics, where an applicable rule always enables reduction, or the reduction follows from the inductive hypothesis (Carbone and Montesi, 2013; Cruz-Filipe and Montesi, 2017; Pohjola et al., 2022). However, a typing system might still be desirable, as it can impose additional discipline on choreography, such as ensuring correct protocol implementation.

Channel Choreography (ChC), introduced by Carbone and Montesi (2013), is a rich choreography language in which a program consists of roles, threads, and sessions that implement communication protocols. Its semantics guarantee deadlock freedom, while the typing system ensures the correct implementation of protocols by sessions. The typing context comprises three components: a service environment Γ , which stores global types for public channels specifying session execution and local expressions (annotated with threads); a thread environment Θ , which tracks the roles of threads in each session; and a session environment Δ , which stores the types of active sessions.

It ensures that a well-typed choreography, with public channels specified by Γ and threads assuming roles in Θ , maintains disciplined sessions governed by Δ . Furthermore, runtime typing introduces a delegation environment to handle changes in the typing context caused by asynchrony or parallelism, ensuring that the program adheres to the protocol throughout execution.

When local computation is involved in the choreography, a typing system is needed to maintain deadlock freedom, as it is impossible to determine whether local computations will terminate. To the best of our knowledge, the closest related work is Pirouette by Hirsch and Garg (2022). In their approach, they use a substitution model with small-step semantics for the local language. Relying on a set of admissible typing rules to provide a reasoning ground, their results show that type preservation and progress for the assumed local language implies the same properties for the choreography. While their progress result aligns with ours, our local language uses big-step

semantics, meaning that preservation in choreography requires strong normalisation from the local language, rather than just type preservation.

Pirouette is a higher-order functional choreography language with three value types: local values, local functions (mapping local values to choreographies), and global functions (mapping choreographies to choreographies). This structure enables choreographies to return values, forming the basis of its typing system. On the other hand, since neither Kalas nor Kalas enriched with *Sprinkles* has return values for choreographies, our typing system mainly checks for well-formedness of a choreography within the given typing environment and if any local computation is involved, we discipline it with its own typing system in a localised typing environment. Pirouette types its local values in a similar projected typing environment, binding variables to types at a specific location.

Another state-of-art functional choreography language $\text{Chor}\lambda$ by [Cruz-Filipe et al. \(2022\)](#) uses a different approach than Pirouette, where choreographies are interpreted as terms in λ -calculus. $\text{Chor}\lambda$ assumes local values for communications, without considering how they are computed. This results in a distinct typing system: local value types are annotated with roles and are part of the global types, rather than being projected from a global environment. Type preservation and progress for choreographies follow from $\text{Chor}\lambda$'s typing and semantic rules.

3.1.2 Handling Exceptions

Choral is an object-oriented choreography implemented as a Java library ([Giallorenzo et al., 2024](#)). It treats a choreography as a class and implements Endpoint Projection (EPP) as generating role-specific classes from the choreography. Local computation is delegated to Java. Higher-order functional choreography models like Pirouette and $\text{Chor}\lambda$ can be seen as formal model candidates for Choral. Its type checker is similar to the typing system in ChC, with types for public channels specifying the sender and receiver roles, as well as the types of communicated messages, and local types annotated with roles.

In terms of communication failure which may raise an exception when a role is trying to perform operation based on reading from the place where a received message has not arrived yet, Choral implements the failure model in RC by [Montesi and Peressotti \(2017\)](#), using recover strategies such as capped attempts or timer within java try-catch block.

To handle communication failures, where an exception may be raised if a role tries to perform an operation based on reading a message that has not yet arrived, Choral implements the failure model Reliable Choreography (RC) described by [Montesi and Peressotti \(2017\)](#). More specifically, it uses recovery strategies, such as capped retry attempts or timers, within Java's try-catch blocks.

RC is a model that addresses communication failures with recovery strategies for senders and receivers. These strategies include using a while loop to wait until the message is received, ensuring exactly-once delivery in an ideal setting where messages cannot be lost, or using a timer or capped attempts for best-effort delivery. A typing system is designed to ensure almost-once delivery or exactly-once delivery given different settings. RC uses a configuration where both sender and receiver have stacks, initially marked as false with a payload value or as ticked when the payload value is no longer in the stack, indicating a sent or received message. In Kalas enriched with *Sprinkles*, we do not consider message-sending failures caused by message being lost. Instead, communication exceptions can be caused by incorrect message types. We also consider local computation failures and include exceptions for errors such as division by zero. However, these exceptions are mainly recorded by transition labels, we do not implement any recovery strategy and always transition the choreography into $\mathbf{0}$, when an error occurs.

3.2 Approach to Strong Normalisation

Although *Sprinkles* is an extension of the simply typed lambda calculus (STLC), its functional big-step semantics and function closures make the proof of strong normalisation slightly different from the standard approach.

A standard strong normalisation proof for STLC with non-deterministic reductions, small-step semantics, and substitutions would define an inductive notion SN for strong normalisation and prove the notion SN is sound that any term belonging to SN is indeed strongly normalising (Abel et al., 2019).

The definition of SN must address the application of terms by introducing an additional constraint called strong head reduction. This is necessary because if $[N/x]M$ is strongly normalising, it does not imply that either $(\lambda x : A.M)N$ or N is strongly normalising, as there may be no occurrences of x in N . Therefore, strong head reduction requires N to also adhere to the notion of SN.

However, *Sprinkles* uses a functional big-step semantics as outlined by Owens et al. (2016), leading us to implement our semantics as an evaluation function with clocks. This setup guarantees that the evaluation function of *Sprinkles* is both total and deterministic.

As a result, strong normalisation proof for *Sprinkles* is equivalent to a weak one, and it does not require us to consider the reduction sequences nor need to consider the strong head reduction for application when using a substitution model. Our proof, is more closed to the strong normalisation proof for a simpler call-by-value STLC in HOL repository by Owens and Laurent (2016), in a sense that they both adopt a functional big-step semantics and use an environment to store bindings for free variables, but *Sprinkles* has richer data types compared to the STLC in HOL example.

As a result, the strong normalisation proof for *Sprinkles* is equivalent to a weak one, without the need to consider reduction sequences or strong head reduction for applications when using a substitution model. Our proof is closer to the strong normalisation proof for the simpler call-by-value STLC found in the HOL repository by Owens and Laurent (2016). Both adopt functional big-step semantics and use an environment to store bindings for free variables, although *Sprinkles* supports a richer set of data types compared to the STLC in the HOL example.

Sprinkles: A Simply Typed Lambda Calculus

We introduce **Sprinkles**, a language over expressions that closely resembles a simply typed lambda calculus. We adopt the functional big-step semantics described by [Owens et al. \(2016\)](#), which guarantees that our evaluation function is both total and deterministic. Our implementation includes an evaluation function that utilises clocks to evaluate expressions, yielding either values or exceptions. We also account for common exceptions, such as division by zero and improperly formatted strings during integer-string conversions.

We use an environment, denoted as E , to store values for free variables. Thus we need to pass E as an argument to our evaluation function. While a substitution model may be mathematically appealing, our environment-based approach aligns well with functional big-step semantics and is relatively easy to implement. This setup leads us to define a typing environment, G , which stores types for free variables. Consequently, expressions in **Sprinkles** are always typed within some typing environment.

When it comes to formalising the type soundness statement for **Sprinkles**, we want to ensure that a typed expression is evaluated in a *correct* environment E according to the typing environment G that types the expression, rather than an arbitrary one. To achieve this, we define $\text{envtype } G \ E$ to characterise such a *correct* environment E in relation to a typing environment G .

4.1 Syntax

Definition 3 (Sprinkles syntax). Values and expressions in *Sprinkles*, ranged over by v, e , are inductively defined by the grammar

$$\begin{aligned}
\text{result} &::= \text{Value } v \mid \text{TypeError} \mid \text{Exn } \textit{exn} \mid \text{Timeout} \\
v &::= \text{IntV } n \mid \text{StrV } s \mid \text{BoolV } b \mid \text{Clos } s \ e \ E \mid \text{PairV } v_1 \ v_2 \mid \text{SumLV } v \mid \text{SumRV } v \\
bop &::= \text{Add} \mid \text{Concat} \mid \text{Mult} \mid \text{Div} \mid \text{Mod} \mid \text{Less} \mid \text{And} \mid \text{Or} \mid \text{Eq} \mid \text{Sub} \mid \text{Pair} \\
uop &::= \text{Not} \mid \text{NumOf} \mid \text{StrOf} \mid \text{Fst} \mid \text{Snd} \mid \text{SumL} \mid \text{SumR} \\
e &::= \begin{array}{lll} \text{Var } x & (var) & \text{StrLit } s \quad (str) \\ \text{IntLit } n & (int) & \text{BoolLit } b \quad (bool) \\ \text{BinOp } bop \ e_1 \ e_2 & (bop) & \text{Uop } uop \ e \quad (uop) \\ \text{If } bg \ e_1 \ e_2 & (if) & \text{Let } x \ e_1 \ e_2 \quad (let) \\ \text{Fn } x \ e & (fn) & \text{App } e_1 \ e_2 \quad (app) \\ \text{Case } e \ x \ e_1 \ y \ e_2 & (case) & \end{array}
\end{aligned}$$

$\text{Var } x$ represents a variable where x is the string that represents variable's name. $\text{BinOp } bop \ e_1 \ e_2$ represents a binary operation and $\text{Uop } uop \ e$ represents unary operation. We include literals for string, integer and boolean in our syntax to allow expressions such as $\text{BinOp Add (Var } x) (\text{IntLit } 1)$. If , Let , Fn , and App are standard. $\text{Case } e \ x \ e_1 \ y \ e_2$ represents a sum case expression, where the bound variable x and expression e_1 correspond to the left branch, while the bound variable y and e_2 correspond to the right branch. Free variables for *Sprinkles*' expressions are standard.

We separate the syntax definitions for binary and unary operations from the concrete definitions of specific binary and unary operators. This makes it easy to extend the set of operators: to add a new operator, we only need to update the bop or uop definitions and provide the corresponding evaluation rule. This approach also significantly reduces the number of cases in proofs that depend on the syntactic structure of expressions, as there's just a single case for all binary operators and a single case for all unary operators.

We include exceptions to handle division by zero and bad formatted integer strings. They are thrown by the corresponding operators. We define three observable evaluation results: as either values, or we encounter a type error (for example, $\text{BinOp Add (IntLit } 1) (\text{StrLit "1"})$), or we reach an exception, or the evaluation times out for the given clock value.

We include exceptions to handle cases like division by zero and improperly formatted integer strings, which are raised by the relevant operators. We define four observable outcomes for evaluation: a successful value v , a **TypeError** for evaluating expressions like $\text{BinOp Add (IntLit } 1) (\text{StrLit "1"})$, an exception $\text{Exn } \textit{exn}$, or a **Timeout** if the evaluation exceeds the specified clock limit.

4.2 Statics

Our typing rules for expressions closely follow those of a simply typed lambda calculus. Because values are not part of the expression syntax, we also define a separate typing relation for values.

Our expressions are typed in a typing environment G which stores mappings from variable names to types. Similar to a variable expression is evaluated by looking up variable name in the given environment E , a variable expression is typed by looking up type stored under the variable name in the given typing environment G . This setup, requires us to constrain the typing environment G and dynamic environment E in the type soundness statement to make sure we are evaluating the typed expression using a correct dynamic environment that aligns with the typing constrain specified in G .

Our expressions are typed within a typing environment, G , which maps variable names to types. Just as a variable expression is evaluated by looking up its variable name in the dynamic environ-

ment E , a variable expression is typed by checking the type associated with that variable name in the typing environment G . This setup requires the dynamic environment E to be aligned with the types specified in the typing environment G in the type soundness statement, so we can ensure expressions are evaluated in a correct environment. We formalise this constrain in Section 4.2.1.

For typing binary and unary operations, we delegate the task of checking operand types against a specific operator to two external typing relations, **boptype** and **uoptype**. This approach allows us to define just two general rules for binary and unary operations when specifying typing rules for expressions. This results in fewer cases in the type soundness proof for operators, requiring only two separate soundness lemmas for **boptype** and **uoptype**. Our design showcases a clear separation of concerns.

4.2.1 Syntax and Typing Rules

Definition 4 (Types of Sprinkles). $t ::= \text{intT} \mid \text{strT} \mid \text{boolT} \mid \text{fnT } t \ t \mid \text{pairT } t \ t \mid \text{sumT } t \ t$

Definition 5 (Typing rules for Sprinkles).

$$\begin{array}{c}
\text{T-CASE} \frac{G \vdash_s e : (\text{sumT } t_1 \ t_2) \quad G[s_1 := t_1] \vdash_s e_1 : ty \quad G[s_2 := t_2] \vdash_s e_2 : ty}{G \vdash_s \text{Case } e \ s_1 \ e_1 \ s_2 \ e_2 : ty} \\
\\
\text{T-IF} \frac{G \vdash_s e_1 : \text{boolT} \quad G \vdash_s e_2 : ty \quad G \vdash_s e_3 : ty}{G \vdash_s \text{If } e_1 \ e_2 \ e_3 : ty} \\
\\
\text{T-BINOP} \frac{\text{boptype } bop \ t_1 \ t_2 \ ty \quad G \vdash_s e_1 : t_1 \quad G \vdash_s e_2 : t_2}{G \vdash_s \text{BinOp } bop \ e_1 \ e_2 : ty} \\
\\
\text{T-VAR} \frac{G \ x = ty}{G \vdash_s \text{Var } x : ty} \quad \text{T-LET} \frac{G \vdash_s e_1 : ety \quad G[s := ety] \vdash_s e_2 : ty}{G \vdash_s \text{Let } s \ e_1 \ e_2 : ty} \\
\\
\text{T-FN} \frac{G[s := t] \vdash_s e : ty}{G \vdash_s \text{Fn } s \ e : (\text{fnT } t \ ty)} \quad \text{T-APP} \frac{G \vdash_s e_1 : (\text{fnT } t \ ty) \quad G \vdash_s e_2 : t}{G \vdash_s \text{App } e_1 \ e_2 : ty}
\end{array}$$

Expressions

Typing statements for expressions are formulated as $G \vdash_s e : t$, where t is the type of expression e given the typing environment G . A variable x is typed by looking it up in G . The typing rules for other expressions are standard.

To type a binary operation $\text{BinOp } bop \ e_1 \ e_2$, we need the types of e_1 and e_2 , given G , to satisfy the constraints defined by **boptype** for the operator bop . The typing rules for unary operations are defined in a similar manner.

Environments

Before we define types for values, we first introduce **envtype** $G \ E$ as a typing constraint that relates the typing environment G to the dynamic environment E . Specifically, **envtype** $G \ E$ requires that the values of variables stored in E conform to the types of variables specified in G .

Definition 6 (Typing constrains for environments). *Given a typing environment G and a dynamic environment E ,*

$$\begin{aligned} \text{envtype } G \ E &\iff \\ \forall \text{str } ty. G \ \text{str} = ty &\Rightarrow \exists v. E \ \text{str} = v \wedge \vdash_s v : ty \end{aligned}$$

Values

Definition 7 (Typing rules for values).

$$\begin{array}{c} \text{T-FN} \frac{\forall \text{str } ty. G \ \text{str} = ty \Rightarrow \exists v. E \ \text{str} = v \wedge \vdash_s v : ty \quad G[s := t_1] \vdash_s e : t_2}{\vdash_s \text{Clos } s \ e \ E : \text{fnT } t_1 \ t_2} \\[10pt] \text{T-PAIR} \frac{\vdash_s v_1 : t_1 \quad \vdash_s v_2 : t_2}{\vdash_s \text{PairV } v_1 \ v_2 : \text{pairT } t_1 \ t_2} \quad \text{T-SUML} \frac{\vdash_s v : t_1}{\vdash_s \text{SumLV } v : \text{sumT } t_1 \ t_2} \\[10pt] \text{T-SUMR} \frac{\vdash_s v : t_2}{\vdash_s \text{SumRV } v : \text{sumT } t_1 \ t_2} \quad \text{T-BOOLV} \frac{}{\vdash_s \text{BoolV } b : \text{boolT}} \end{array}$$

In $\vdash_s v : t$, a value v is related to a type t . Literal value cases are obvious. Typings for pair and sum values are standard. Closure value, represented as $\text{Clos } s \ e \ E$, is a triple (s, e, E) corresponding to a function expression $\text{Fn } s \ e$. In this closure, s represents the function parameter, e is the function body, and E is the environment that stores the bindings of free variables when the function definition is evaluated.

To type a closure value $\text{Clos } s \ e \ E$, if G accurately describes the types of variable values stored in E , we require that e has the output type t_2 in $G[s := t_1]$. The constraints for G and E align with Definition 6.¹

Requiring the constraint $\text{envtype } G \ E$ in the definition of $\vdash_s \text{Clos } s \ e \ E : \text{fnT } t_1 \ t_2$ is appropriate because, when the closure value is applied, e will be evaluated in the extended closure environment $E[s := v]$ for some argument value v . Therefore, we want to have e typed within a typing environment that accurately describes the types of values stored in $E[s := v]$. As we will see in Section 4.4.2, the constraint $\text{envtype } G \ E$ is also necessary for the proof of the $\text{App } e_1 \ e_2$ case in the type soundness proof.

As a consequence of Definition 7 and 5, $\text{SumLV } v$ does not have a unique type because t_2 is arbitrary, and the same applies to $\text{SumRV } v$. Furthermore, the function $\text{Fn } s \ (\text{Var } s)$ also has non-unique types, as the variable s can take on arbitrary types.

4.2.2 Typing Examples

Example 4.3.2 discussed in Section 4.3.1 to show our function closure supports currying can be typed using the rules in Definition 5. However, the Y combinator discussed in Section 4.3.1 cannot be typed. If we try to derive the type of Y combinator by inference, we will reach recursively defined type variables that are not solvable.

¹We do not apply $\text{envtype } G \ E$ directly in the closure case because the definition of envtype depends on $\vdash_s$, and we want to avoid mutually recursive definitions between a definition and a relation in HOL4.

4.3 Dynamics

We adopt a functional big-step semantics. As a result, our semantics are implemented as an evaluation function that takes a clock value c , an environment E , and an expression e as inputs. We use clocks since the size of expressions are decreasing except for the application case, so decreasing the clock value when evaluation an application ensures the termination of our evaluation function. Our definition of evaluation function also implies a call-by-value evaluation strategy.

We define the evaluation function as:

Definition 8 (Evaluation rules for Sprinkles).

$$\begin{aligned}
& \text{eval_exp } c \ E \ (\text{Var } str) \stackrel{\text{def}}{=} \\
& \quad \text{case } E \ str \text{ of None} \Rightarrow \text{TypeError} \mid \text{Some } v \Rightarrow \text{Value } v \\
& \text{eval_exp } c \ E \ (\text{StrLit } str) \stackrel{\text{def}}{=} \text{Value } (\text{StrV } str) \\
& \text{eval_exp } c \ E \ (\text{BinOp } bop \ e_1 \ e_2) \stackrel{\text{def}}{=} \\
& \quad \text{do} \\
& \quad \quad v_1 \leftarrow \text{eval_exp } c \ E \ e_1; \\
& \quad \quad v_2 \leftarrow \text{eval_exp } c \ E \ e_2; \\
& \quad \quad \text{eval_bop } bop \ v_1 \ v_2 \\
& \quad \text{od} \\
& \text{eval_exp } c \ E \ (\text{If } e_1 \ e_2 \ e_3) \stackrel{\text{def}}{=} \\
& \quad \text{do} \\
& \quad \quad v \leftarrow \text{eval_exp } c \ E \ e_1; \\
& \quad \quad b \leftarrow \text{getB } v; \\
& \quad \quad \text{if } b \text{ then eval_exp } c \ E \ e_2 \text{ else eval_exp } c \ E \ e_3 \\
& \quad \text{od} \\
& \text{eval_exp } c \ E \ (\text{Let } s \ e_1 \ e_2) \stackrel{\text{def}}{=} \\
& \quad \text{do } v_1 \leftarrow \text{eval_exp } c \ E \ e_1; \text{ eval_exp } c \ E[s := v_1] \ e_2 \text{ od} \\
& \text{eval_exp } c \ E \ (\text{App } e_1 \ e_2) \stackrel{\text{def}}{=} \\
& \quad \text{if } c > 0 \text{ then} \\
& \quad \quad \text{do} \\
& \quad \quad \quad v_1 \leftarrow \text{eval_exp } c \ E \ e_1; \\
& \quad \quad \quad v_2 \leftarrow \text{eval_exp } c \ E \ e_2; \\
& \quad \quad \quad \text{case } v_1 \text{ of} \\
& \quad \quad \quad \quad \text{Clos } s \ e \ E_1 \Rightarrow \text{eval_exp } (c - 1) \ E_1[s := v_2] \ e \\
& \quad \quad \quad \quad \mid _ \Rightarrow \text{TypeError} \\
& \quad \quad \quad \text{od} \\
& \quad \quad \text{else Timeout} \\
& \text{eval_exp } c \ E \ (\text{Case } e \ s_1 \ e_1 \ s_2 \ e_2) \stackrel{\text{def}}{=} \\
& \quad \text{do} \\
& \quad \quad v \leftarrow \text{eval_exp } c \ E \ e; \\
& \quad \quad \text{case } v \text{ of} \\
& \quad \quad \quad \text{SumLV } v_0 \Rightarrow \text{eval_exp } c \ E[s_1 := v_0] \ e_1 \\
& \quad \quad \quad \mid \text{SumRV } v'_0 \Rightarrow \text{eval_exp } c \ E[s_2 := v'_0] \ e_2 \\
& \quad \quad \quad \mid _ \Rightarrow \text{TypeError} \\
& \quad \quad \text{od}
\end{aligned}$$

Rules for If, Let, and Case are standard. We evaluate $\text{Var } x$ by looking up the variable x in the provided environment E . If E does not contain a binding for x , it indicates that x is an undefined

variable. Thus `TypeError` is returned.

For `Bin bop e1 e2`, we first evaluate `e1` and `e2` to obtain `v1` and `v2`. Next, we use `eva_bop bop v1 v2` to compute the result based on the specific rule for the operator `bop`, which is defined in `eval_bop`. The evaluations for unary operations are defined in a similar manner. The definitions of `eval_uop` and `eval_bop` can be found in Appendix A.

Based on the rules outlined so far, let's consider an evaluation example of the expression `Uop NumOf (Var x)`. The function `eval_exp c E (Uop NumOf (Var x))` will first compute

$$\text{eval_exp } c \ E \ (\text{Var } x)$$

by looking up the variable `x` in the environment `E`. If `x` is not found in `E`, the function will return `TypeError`. If the lookup succeeds, `eval_exp c E (Var x)` will return a value `v`, and we will then compute `eval_uop NumOf v`. If `v` is not a correctly formatted integer string (for instance, if it contains a non-numeric character), `eval_uop NumOf v` will return `Exn BadStr`. Otherwise, `eval_uop NumOf v` will return the converted integer value `IntV n`. The relevant format checking and string conversion algorithms are utilised by `eval_uop` during the evaluation process.

4.3.1 Function Closure

We first define $f \downarrow_s$.

Definition 9 (Restricting a finite map). *Given a finite map f and a set r , we define $f \downarrow_r$ as*

$$\begin{aligned} \text{domain}(f \downarrow_r) &= \text{domain}(f) \cap r \wedge \\ \forall x. f \downarrow_r ' x &= \text{if } x \in \text{domain}(f) \cap r \text{ then } f ' x \text{ else } \emptyset ' x \end{aligned}$$

Then we give the evaluation rule for `Fn s e`:

Definition 10 (Evaluation rule for function).

$$\text{eval_exp } c \ E \ (\text{Fn } s \ e) \stackrel{\text{def}}{=} \text{Value } (\text{Clos } s \ e \ (E \downarrow_{\text{fv}(e)} \setminus s))$$

However, our rule stores a restricted environment `Clos s e (E  $\downarrow_{\text{fv}(e)}$   $\setminus$  s)` in the evaluated closure, which may seem unexpected. Nonetheless, we argue that `Clos s e (E  $\downarrow_{\text{fv}(e)}$   $\setminus$  s)` behaves the same as `Clos s e E` during evaluations and will produce the same result.

First, $E \downarrow_{\text{fv}(e)}$ includes values for all free variables in `e`. Second, excluding `s` is safe because when we apply the restricted closure to a value, we always bind `s` to the argument value. So the previous value of `s` stored in the environment becomes irrelevant. This modification is also necessary to preserve Proposition 3. Further discussions can be found in Section 6.6.1.

By using a closure, we can keep track of the values for bound variables in nested function definitions. If we do not record the environment when a function definition is evaluated, we risk losing the bindings for all other bound variables from outer scopes once we complete the evaluation of the innermost function definition. Additionally, currying would be impossible without utilising a closure, as illustrated in Example 4.3.1.

Example 4.3.1. Consider evaluating the following expression in an empty environment:

1. `eval_exp c  $\emptyset$  (App (App (Fn x (Fn y (BinOp Add (Var x) (Var y)))) (IntLit 1)) (IntLit 2))`
2. $\longrightarrow \text{eval_exp } c \ \emptyset \ (\text{App } (\text{Fn } x \ (\text{Fn } y \ (\text{BinOp Add } (\text{Var } x) \ (\text{Var } y)))) \ (\text{IntLit } 1))$
3. $\longrightarrow \text{eval_exp } c \ \emptyset \ (\text{Fn } x \ (\text{Fn } y \ (\text{BinOp Add } (\text{Var } x) \ (\text{Var } y)))) \Downarrow (x, \text{Fn } y \ (\text{BinOp Add } (\text{Var } x) \ (\text{Var } y)))$

4. $\longrightarrow_{\text{eval_exp}} c \ \emptyset \ (\text{IntLit } 1) \Downarrow \text{IntV } 1$
5. $\longrightarrow_{\text{eval_exp}} (c - 1) \ \emptyset[x := \text{IntV } 1] \ (\text{Fn } y \ (\text{BinOp Add } (\text{Var } x) (\text{Var } y))) \Downarrow$
 $(y, \text{BinOp Add } (\text{Var } x) (\text{Var } y))$
6. $\longrightarrow_{\text{eval_exp}} c \ \emptyset \ (\text{IntLit } 2) \Downarrow \text{IntV } 2$
7. $\longrightarrow_{\text{eval_exp}} (c - 1) \ \emptyset[y := \text{IntV } 2] \ (\text{BinOp Add } (\text{Var } x) (\text{Var } y)) \Downarrow ?$

When we are evaluating $\text{App } (\text{Fn } x \ (\text{Fn } y \ (\text{BinOp Add } (\text{Var } x) (\text{Var } y)))) \ (\text{IntLit } 1)$ in line 5 we know x is bound to 1. But we lose this information when we store the result of line 5 as

$$(y, \text{BinOp Add } (\text{Var } x) (\text{Var } y)).$$

So we only know y is bound to $\text{IntV } 2$ when we try to evaluate $\text{BinOp Add } (\text{Var } x) (\text{Var } y)$ in line 7, which fails since the value of x is unknown.

By using function closure to record values of all bound variables when a function definition is being evaluated, we are able to evaluate $\text{BinOp Add } (\text{Var } x) (\text{Var } y)$, as illustrated in line 7 of Example 4.3.2. It is possible that a function body contains unbound free variables, but such expression cannot be typed in an empty environment and it is reasonable to make its evaluation fails.

Example 4.3.2. Consider evaluating the following expression in an empty environment:

1. $\text{eval_exp } c \ \emptyset \ (\text{App } (\text{App } (\text{Fn } x \ (\text{Fn } y \ (\text{BinOp Add } (\text{Var } x) (\text{Var } y)))) \ (\text{IntLit } 1)) \ (\text{IntLit } 2))$
2. $\longrightarrow_{\text{eval_exp}} c \ \emptyset \ (\text{App } (\text{Fn } x \ (\text{Fn } y \ (\text{BinOp Add } (\text{Var } x) (\text{Var } y)))) \ (\text{IntLit } 1))$
3. $\longrightarrow_{\text{eval_exp}} c \ \emptyset \ (\text{Fn } x \ (\text{Fn } y \ (\text{BinOp Add } (\text{Var } x) (\text{Var } y)))) \Downarrow$
 $\text{Clos } x \ (\text{Fn } y \ (\text{BinOp Add } (\text{Var } x) (\text{Var } y))) \ \emptyset$
4. $\longrightarrow_{\text{eval_exp}} c \ \emptyset \ (\text{IntLit } 1) \Downarrow \text{IntV } 1$
5. $\longrightarrow_{\text{eval_exp}} (c - 1) \ \emptyset[x := \text{IntV } 1] \ (\text{Fn } y \ (\text{BinOp Add } (\text{Var } x) (\text{Var } y))) \Downarrow$
 $\text{Clos } y \ (\text{BinOp Add } (\text{Var } x) (\text{Var } y)) \ \emptyset[x := \text{IntV } 1]$
6. $\longrightarrow_{\text{eval_exp}} c \ \emptyset \ (\text{IntLit } 2) \Downarrow \text{IntV } 2$
7. $\longrightarrow_{\text{eval_exp}} (c - 1) \ \emptyset[x := \text{IntV } 1][y := \text{IntV } 2] \ (\text{BinOp Add } (\text{Var } x) (\text{Var } y)) \Downarrow \text{IntV } 3$

Recursion

Sprinkles's syntax supports recursion, as one can write a Y combinator

$$\text{Fn } f \ (\text{App } (\text{Fn } x \ (\text{App } (\text{Var } f) (\text{App } (\text{Var } x) (\text{Var } x)))) \ (\text{Fn } x \ (\text{App } (\text{Var } f) (\text{App } (\text{Var } x) (\text{Var } x))))).$$

However, $\text{Let } f \ e_1 \ e_2$ does not support recursion because f is not bound within e_1 . Additionally, our typing system is unable to type the Y combinator.

We propose that recursion can be easily and effectively integrated into Sprinkles by introducing the syntax **Letric**. The expression $\text{Letric } f \ s \ e_1 \ e_2$ will be evaluated into a quadruple (f, x, e, E) . This quadruple is similar to the one described by [Hardin et al. \(2021\)](#).

4.3.2 Main Properties

Functional big-step semantics ensures that our evaluation function is total and deterministic by design. With this in mind, we define the evaluation of an expression as *terminating* if there exists a clock value for which the evaluation function returns a result that is not **Timeout**. Conversely, if the evaluation returns **Timeout** for all clock values, we say that the evaluation of the expression *diverges*.

The evaluation of **Sprinkles** is also deterministic in the sense that if the evaluation for an expression terminates with an initial clock value, then increasing the clock will always yield the same result. We demonstrate this result as a corollary that follows from Lemmas 1, 2, and 3.

The first lemma states that if the expression e evaluates to a value v in the environment E with a clock value c_0 , then evaluating the same expression in the same environment with a larger clock value c_1 will also yield the same value v . The statements for Lemmas 2 and 3 are analogous.

Lemma 1 (Increasing clock for value results). *Given a initial clock c_0 , an expression e , and an environment E , we have*

$$\begin{aligned} \text{eval_exp } cl_0 \ E \ e &= \text{Value } v \Rightarrow \\ \forall cl_1. cl_1 \geq cl_0 &\Rightarrow \text{eval_exp } cl_1 \ E \ e = \text{Value } v \end{aligned}$$

Proof: Using the well-founded lexicographic induction principle by Definition 8, results for each case directly follows the inductive hypotheses. The induction principle can be found in Appendix A. $\square$

Lemma 2 (Increasing clock for exception results). *Given a initial clock c_0 , an expression e , and an environment E , we have*

$$\begin{aligned} \text{eval_exp } cl_0 \ E \ e &= \text{Exn } exn \Rightarrow \\ \forall cl_1. cl_1 \geq cl_0 &\Rightarrow \text{eval_exp } cl_1 \ E \ e = \text{Exn } exn \end{aligned}$$

Proof: The proof relies on the same induction principle for Lemma 1 and depends on Lemma 1, since in the inductive case an exception can occur after a value. $\square$

Lemma 3 (Increasing clock for type error results). *Given a initial clock c_0 , an expression e , and an environment E , we have*

$$\begin{aligned} \text{eval_exp } cl_0 \ E \ e &= \text{TypeError} \Rightarrow \\ \forall cl_1. cl_1 \geq cl_0 &\Rightarrow \text{eval_exp } cl_1 \ E \ e = \text{TypeError} \end{aligned}$$

Proof: Similar to the proof for Lemma 2. $\square$

Corollary 1 (Determinism). *Given a initial clock c_0 , an expression e , an environment E , and a result r , we have*

$$\begin{aligned} \text{eval_exp } cl_0 \ E \ e &= r \wedge r \neq \text{Timeout} \Rightarrow \\ \forall cl_1. cl_1 \geq cl_0 &\Rightarrow \text{eval_exp } cl_1 \ E \ e = r \end{aligned}$$

Proof: Cases analyses based on the syntax form of r . Each case follows either Lemma 1, 2, or 3. $\square$

Corollary 1 is important for the proof of strong normalisation. For example, in the case of the if statement, we know that the evaluation of the boolean guard and the two branches terminates by the inductive hypotheses for their respective clocks. We need to ensure that there exists a common clock value on which the evaluation of all three expressions also terminates and returns the same results. Further discussion on the use of Corollary 1 in the strong normalisation proof can be found in Section 5.3.2.

4.4 Type Soundness

4.4.1 Lemmas

We will discuss several lemmas and propositions for the proposed semantics and typing rules before proceeding with the type soundness proof.

Our type soundness proof is similar to the type soundness proof for a simply typed lambda calculus by [Pierce \(2002\)](#) in a sense that we require, or at least require an equivalent version of, the generation lemma, the lemma of canonical forms of types, and the type preservation under substitution lemma used in [Pierce \(2002\)](#)'s proof.

Our type soundness proof is similar to the type soundness proof for a simply typed lambda calculus with small-step semantics and the substitution model, such as the one presented by [Pierce \(2002\)](#). Specifically, we require, or at least need an equivalent version of, the generation lemma, the lemma on canonical forms of types, and the lemma on type preservation under substitution, all of which are utilised in [Pierce \(2002\)](#)'s proof.

Lemma 4 (Generation lemma for Sprinkles). *Given a typed expression, we have*

1. (Variable)

$$G \vdash_s \text{Var } x : ty \iff G x = ty$$

2. (Binary Operation)

$$\begin{aligned} G \vdash_s \text{BinOp } bop \ e_1 \ e_2 : ty &\iff \\ \exists t_1 \ t_2. \text{boptype } bop \ t_1 \ t_2 \ ty \ \wedge \ G \vdash_s \ e_1 : t_1 \ \wedge \ G \vdash_s \ e_2 : t_2 \end{aligned}$$

3. (Let)

$$\begin{aligned} G \vdash_s \text{Let } s \ e_1 \ e_2 : ty &\iff \\ \exists ety. G \vdash_s \ e_1 : ety \ \wedge \ G[s := ety] \vdash_s \ e_2 : ty \end{aligned}$$

4. (Function)

$$\begin{aligned} G \vdash_s \text{Fn } s \ e : ty &\iff \\ \exists t \ ty'. ty = \text{fnT } t \ ty' \ \wedge \ G[s := t] \vdash_s \ e : ty' \end{aligned}$$

5. (Application)

$$G \vdash_s \text{App } e_1 \ e_2 : ty \iff \exists t. G \vdash_s \ e_1 : (\text{fnT } t \ ty) \ \wedge \ G \vdash_s \ e_2 : t$$

Proof: Immediately follows from Definition 5. □

The generation lemma enables us to advance one step in the type derivation tree from the bottom to the top. It allows us to obtain types and typing information for the sub-expressions within a well-typed expression.

Lemma 5 (Canonical forms for typed values).

1. (Boolean)

$$\vdash_s v : \text{boolT} \iff \exists b. v = \text{BoolV } b$$

2. (Closure)

$$\begin{aligned} \vdash_s v : \text{fnT } ty_1 \ ty_2 &\iff \\ \exists s \ e \ E \ G. & \\ v = \text{Clos } s \ e \ E \ \wedge \ \text{envtype } G \ E \ \wedge \ G[s := ty_1] \vdash_s \ e : ty_2 \end{aligned}$$

3. (Sum)

$$\begin{aligned} \vdash_s v : \text{sumT } t_1 \ t_2 &\iff \\ \exists v_0. v = \text{SumLV } v_0 \ \wedge \ \vdash_s v_0 : t_1 \ \vee \ v = \text{SumRV } v_0 \ \wedge \ \vdash_s v_0 : t_2 \end{aligned}$$

Proof: Immediately follows from Definition 11. $\square$

The canonical form lemma provides the concrete syntax of a value v along with the typing information for the expressions involved in that concrete syntax, given that we have $\vdash_s v : t$. This is particularly useful for inductive cases in the type soundness proof, as the inductive hypothesis tells us that expressions evaluate to a value v and that $\vdash_s v : t$ holds. Moreover, the concrete syntax of v always aligns with the expressions in goal.

The type preservation under substitution lemma used by Pierce (2002) states that substituting variables with terms of appropriate types preserves the original type. Since we evaluate variables by looking them up in an environment E , the concept of `envtype` G E (as defined in Definition 6) is analogous to the type preservation under substitution lemma.

As discussed in Section 4.2.1 and Section 4.3, we first evaluate the argument expressions into values, and then we delegate the evaluation of binary and unary operations to `eval_bop` and `eval_uop`. Similarly, we first type the argument expressions and then check the input and output types against the concrete operators using separate typing relations, `botype` and `uotype`. This approach leads to two important properties: if the argument values are properly typed and the operator types match the argument types, then `eval_bop` and `eval_uop` will always output a value of the correct type or result in an exception. We refer to these properties as type soundness for `eval_bop` and `eval_uop`.

Proposition 1 (Type soundness for `eval_bop`). *Given values v_1, v_2 , types t_1, t_2 , and a binary operator bop , we have*

$$\begin{aligned} \vdash_s v_1 : t_1 \wedge \vdash_s v_2 : t_2 \wedge \text{botype } bop \ t_1 \ t_2 \ ty \Rightarrow \\ (\exists v. \text{eval_bop } bop \ v_1 \ v_2 = \text{Value } v \wedge \vdash_s v : ty) \vee \\ \exists e. \text{eval_bop } bop \ v_1 \ v_2 = \text{Exn } e \end{aligned}$$

Proof: `botype` is non-recursive. We prove by performing cases analyses based on `botype` rules and each case follows from Lemma 5. $\square$

Proposition 2 (Type soundness for `eval_uop`). *Given a values v , a type t , and a unary operator uop , we have*

$$\begin{aligned} \vdash_s v : t_1 \wedge \text{uotype } uop \ t_1 \ ty \Rightarrow \\ (\exists v'. \text{eval_uop } uop \ v = \text{Value } v' \wedge \vdash_s v' : ty) \vee \\ \exists e. \text{eval_uop } uop \ v = \text{Exn } e \end{aligned}$$

Proof: Same as the above. $\square$

Proposition 1 and Proposition 2 are required for the binary and unary operator cases in the type soundness proof. By the inductive hypothesis, we will have well-typed evaluated argument values, and we need to show that `eval_bop` or `eval_uop` either outputs a well-typed value or results in an exception.

There is also a lemma that allows us to update G and E with a typed value and re-establish the `envtype` property:

Lemma 6 (Updating environments with typed value). *Given a value v of type t , we have*

$$\vdash_s v : ty \Rightarrow \forall G \ E \ s. \text{envtype } G \ E \Rightarrow \text{envtype } G[s := ty] \ E[s := v]$$

Proof: Since the typing rules for values (Definition 7) are defined inductively, we will prove them by induction on Definition 7. The base cases include integer, string, and boolean literals, which directly follow from the definition of `envtype` (Definition 6). The cases for pair and sum

values derive directly from the inductive hypothesis. The closure value case does not rely on the inductive hypothesis; instead, it follows from the closure case in the lemma of canonical forms of types (Lemma 5). $\square$

This lemma is useful in the function, application, and sum cases of the type soundness proof, where we have sub-expressions within the goal expression that are typed in an extended typing environment. We need to re-establish the `envtype` property for both the extended typing environment and the dynamic environment so that we can apply the results from the corresponding inductive hypothesis.

Restricted Function Closure

Since we evaluate a function expression into a restricted closure value (as discussed in Section 4.3.1), we will need the following lemmas for the function case in the type soundness proof. Given that $G \vdash_s \text{Fn } s \ e : (\text{fnT } t_1 \ t_2)$, Lemma 4 ensures that $G[s := t_1] \vdash_s e : t_2$. From the premise of the goal, we have `envtype` $G \ E$. However, to show that

$$\vdash_s \text{Clos } s \ e \ (E \upharpoonright_{\text{fv}(e)} \setminus s) : \text{fnT } t_1 \ t_2,$$

we need to show that there exists a typing environment G' such that `envtype` $G' \ (E \upharpoonright_{\text{fv}(e)} \setminus s)$ holds (Definition 7). Clearly, the original G does not qualify, as its domain may be larger than that of $E \upharpoonright_{\text{fv}(e)} \setminus s$. Thus, Corollary 2, which we will prove shortly, provides us with a minimal typing environment G' in which e has type t_2 and for which `envtype` $G' \ (E \upharpoonright_{\text{fv}(e)} \setminus s)$ holds, thereby proving the goal.

The first lemma states that if an expression e is typed in an environment G , then G contains type information for all free variables of e .

Lemma 7. *Given an expression e of type t , typed within the typing environment G , we have*

$$G \vdash_s e : ty \Rightarrow \text{fv}(e) \subseteq \text{domain}(G)$$

Proof: By rule induction on Definition 5. $\square$

The second lemma states that if an expression e is typed in an environment $G1$, then e retains the same type in a smaller typing environment $G2$, provided that $G2$ contains type information for all free variables in e .

Lemma 8 (Reducing typing environment). *Given an expression e of type t , typed within the typing environment $G1$, we have*

$$G1 \vdash_s e : ty \Rightarrow \forall G2. \text{fv}(e) \subseteq \text{domain}(G2) \wedge G2 \sqsubseteq G1 \Rightarrow G2 \vdash_s e : ty$$

Proof: By rule induction on Definition 5. $\square$

The third lemma states that we can always type the expression e in a smaller typing environment, provided that the smaller typing environment still includes the bindings for the free variables of e .

Lemma 9 (Restricting typing environment). *Given an expression e of type t , typed within the typing environment $G1$, and a set A , we have*

$$G \vdash_s e : ty \wedge \text{fv}(e) \subseteq A \Rightarrow G \upharpoonright_A \vdash_s e : ty$$

Proof: Follows directly from Lemma 7 and Lemma 8. $\square$

Lemma 9 and Lemma 7 provide the existence of a minimal typing environment for an expression e , containing exactly the bindings for all free variables of e .

Corollary 2 (Minimal typing environment). *Given $G \vdash_s e : t$, there exists G' such that G' is the smallest typing environment in which e has type t , and $\text{domain}(G') = \text{fv}(e)$.*

Proof: By Lemma 9, we know that if there exists some G such that $G \vdash_s e : t$, then we have $G \upharpoonright_{\text{fv}(e)} \vdash_s e : t$. According to Lemma 7, G contains bindings for all free variables in e ; thus, $G \upharpoonright_{\text{fv}(e)}$ contains exactly the bindings for all free variables in e , given Definition 9.

For the reverse direction, suppose there exists a typing environment G' that contains fewer bindings than $G \upharpoonright_{\text{fv}(e)}$, and we have $G' \vdash_s e : t$. By Lemma 7, G' must contain bindings for all free variables in e , leading to a contradiction. $\square$

4.4.2 Type Soundness

We are now ready to prove the type soundness property for Sprinkles.

Theorem 1 (Type soundness for Sprinkles). *Given a typing environment G , a dynamic environment E , and an expression e , we have*

$$\begin{aligned} \text{envtype } G \ E \wedge G \vdash_s e : ty \Rightarrow \\ (\exists v. \text{eval_exp } c \ E \ e = \text{Value } v \wedge \vdash_s v : ty) \vee \\ (\exists \text{exn}. \text{eval_exp } c \ E \ e = \text{Exn } \text{exn}) \vee \\ \text{eval_exp } c \ E \ e = \text{Timeout} \end{aligned}$$

Proof: We prove this by induction on the definition of `eval_exp`. The case for `Var` follows directly from Definition 6. In the case of `Let` $s \ e_1 \ e_2$, by Lemma 4, we have $G \vdash_s e_1 : t_0$ and $G[s := t_0] \vdash_s e_2 : t$, given that $G \vdash_s \text{Let } s \ e_1 \ e_2 : t$. Then, by the inductive hypothesis for e_1 , we know that e_1 evaluates to some value v and we have $\vdash_s v : t_0$. Lemma 6 then provides us with $\text{envtype } G[s := t_0] \ E[s := v]$. This allows us to apply the results from the inductive hypothesis to e_2 , thereby proving our goal.

For all following cases, we assume that Lemma 4 is applied first.

For `Fn` $s \ e$ case, given $\text{envtype } G \ E$ and $G[s := t_1] \vdash_s e : t_2$, we need to show that

$$\vdash_s \text{Clos } s \ e \ (E \upharpoonright_{\text{fv}(e)} \setminus s) : \text{fnT } t_1 \ t_2.$$

According to the definitions of `value_type` (Definition 7) and `envtype` (Definition 6), we must demonstrate the existence of a typing environment G_0 such that $G_0[s := t_1] \vdash_s e : t_2$ and $\text{envtype } G_0 \ (E \upharpoonright_{\text{fv}(e)} \setminus s)$.

By Corollary 2, we know there exists a minimal typing environment G' such that

$$G'[s := t_1] \vdash_s e : t_2,$$

and this environment contains bindings for all free variables of e , excluding the binding for s if $s \in \text{fv}(e)$. Therefore, by the definition of `DRESTRICT` (Definition 9) and Definition 6, we have found such an environment G_0 , which is G' .

If we use an unrestricted function closure in Definition 8 instead, we need to show

$$\vdash_s \text{Clos } s \ e \ E : \text{fnT } t_1 \ t_2,$$

which is straightforward given Definition 7.

For **BinOp** and **Uop** cases, results follow directly from Proposition 2 and Proposition 1, as discussed in Section 4.4.1.

For **If** $e \ e_1 \ e_2$ case, the inductive hypothesis for the guard expression e tells us that e evaluates to a value v , where $\vdash_s v : \text{boolT}$. By Lemma 5, we know that v must be either **BoolV T** or **BoolV F**. Both the true and false branches are covered by the inductive hypotheses.

For **App** $e_1 \ e_2$ case, the inductive hypotheses for e_1 and e_2 tell us that they evaluate to v_1 and v_2 , respectively. We also know that $\vdash_s v_1 : \text{fnT } t_1 \ t_2$ and $\vdash_s v_2 : t_1$. By Lemma 5, we conclude that v_1 is a function closure, represented as **Clos** $s \ e \ E$, which matches the expressions in our goal. Additionally, Lemma 5 provides us with $G[s := t_1] \vdash_s e : t_2$, where $\text{envtype } G \ E$. Applying Lemma 6, we obtain $\text{envtype } G[s := t_1] \ E[s := v_2]$. This allows us to utilise the inductive hypothesis for e , which solves the goal.

For **Case** $e \ s_1 \ e_1 \ s_2 \ e_2$ case, the inductive hypothesis tells us that e evaluates to a value v , where $\vdash_s v : \text{sumT } t_1 \ t_2$. By Lemma 5, we know that v is either **SumLV** v' or **SumRV** v' for some value v' .

In both scenarios, we have either $\vdash_s v' : t_1$ or $\vdash_s v' : t_2$. Consequently, by Lemma 6, we find either $\text{envtype } G[s_1 := t_1] \ E[s_1 := v']$ or $\text{envtype } G[s_2 := t_2] \ E[s_2 := v']$. This enables us to apply the inductive hypothesis for e_1 or e_2 , which solves the goal. $\square$

Strong Normalisation

We find that the strong normalisation property of **Sprinkles** is essential for re-establishing the progress property in the enriched choreography. Without it, if local computation in one of the processes fails to terminate, the choreography will deadlock, waiting indefinitely for that process to complete.

The differences between our approach to proving strong normalisation and a more standard method involving simply typed lambda calculus with non-deterministic reductions, small-step semantics, and substitutions are discussed in Section 3.2. In brief, a straightforward induction over Definition 5 fails in the case of **App** $e_1 e_2$, because the e in the function closure **Clos** $s e E$ of e_1 can be arbitrarily large. Instead, we use induction over the types of values, which leads us to define *strongly normalising values*, as introduced in Definition 11.

Sprinkles's evaluation function is deterministic by using a functional big-step semantics, so the strong normalisation proof for **Sprinkles** is equivalent to the weak normalisation proof.

5.1 Formalisation

We now introduce the definitions of logical relations needed to state the Fundamental Lemma.

Strongly Normalising Values

In a standard simply typed lambda calculus, terms can be values, but in **Sprinkles**, values are not part of the expression syntax. Instead, in the recursive cases of our evaluation function (Definition 8), the values of sub-expressions are evaluated first and then used to evaluate the current expression. Therefore, we define $\text{sn.v } t \ v$ as an inductive notion of strongly normalising values, indicating that these values demonstrate strong normalisation behaviour of specific types during evaluation.

Definition 11 (Strongly normalising values).

$$\begin{aligned}
\text{sn_v intT (IntV } n) &\stackrel{\text{def}}{=} \text{T} \\
\text{sn_v strT (StrV } s) &\stackrel{\text{def}}{=} \text{T} \\
\text{sn_v boolT (BoolV } b) &\stackrel{\text{def}}{=} \text{T} \\
\text{sn_v (fnT } t_1 \ t_2) \text{ (Clos } s \ e \ E) &\stackrel{\text{def}}{=} \\
&\forall v. \text{sn_v } t_1 \ v \Rightarrow \\
&(\exists cl \ v'. \\
&\quad \text{eval_exp } cl \ E[s := v] \ e = \text{Value } v' \wedge \text{sn_v } t_2 \ v') \vee \\
&\quad \exists cl \ \text{exn}. \text{eval_exp } cl \ E[s := v] \ e = \text{Exn } \text{exn} \\
\text{sn_v (pairT } t_1 \ t_2) \text{ (PairV } v_1 \ v_2) &\stackrel{\text{def}}{=} \text{sn_v } t_1 \ v_1 \wedge \text{sn_v } t_2 \ v_2 \\
\text{sn_v (sumT } t_1 \ t_2) \text{ (SumLV } v) &\stackrel{\text{def}}{=} \text{sn_v } t_1 \ v \\
\text{sn_v (sumT } t_1 \ t_2) \text{ (SumRV } v) &\stackrel{\text{def}}{=} \text{sn_v } t_2 \ v
\end{aligned}$$

Literal values are always strongly normalising. For a closure value $\text{Clos } s \ e \ E$ of type $\text{fnT } t_1 \ t_2$ to be strongly normalising, we require that if the expression e in $E[s := v]$ is evaluated, given that the value v of type t_1 is strongly normalising, then the evaluation of e must always return a strongly normalising value of type t_2 or result in an exception. A pair value $\text{PairV } v_1 \ v_2$ of type $\text{pairT } t_1 \ t_2$ is strongly normalising if and only if both v_1 and v_2 are strongly normalising. A $\text{SumLV } v$ value of type $\text{sumT } t_1 \ t_2$ is strongly normalising if and only if v is strongly normalising according to type t_1 . The condition for $\text{sn_v (sumT } t_1 \ t_2) \text{ (SumRV } v)$ is defined similarly.

Environments

Since we assign values to free variables by looking them up in an environment during expression evaluation, we introduce a constraint between the typing environment and the dynamic environment. This constraint requires the dynamic environment to store strongly normalising values that correspond to the variables and types specified by the typing environment.

Definition 12 (Environment of strongly normalising typed values). *Given a typing environment G and a dynamic environment E ,*

$$\text{envsn } G \ E \stackrel{\text{def}}{=} \forall s \ ty. G \ s = ty \Rightarrow \exists v. E \ s = v \wedge \text{sn_v } ty \ v$$

Evaluation of Expressions

We define the evaluation of an expression e of type t under the environment E as strongly normalising if and only if, for some initial clock c , the evaluation returns a strongly normalising value or an exception.

Definition 13 (Strong normalisation for evaluation of expressions). *Given an expression e of type t and an environment E ,*

$$\begin{aligned}
\text{sn_exec } t \ E \ e &\stackrel{\text{def}}{=} \\
&(\exists cl \ v. \text{eval_exp } cl \ E \ e = \text{Value } v \wedge \text{sn_v } t \ v) \vee \\
&\exists cl \ \text{exn}. \text{eval_exp } cl \ E \ e = \text{Exn } \text{exn}
\end{aligned}$$

We define the evaluation of an expression e , typed under the typing environment G with type t , as strongly normalising if and only if the evaluation of e is strongly normalising under any dynamic environment E that satisfies $\text{envsn } (G \upharpoonright_{\text{fv}(e)}) \ E$.

Definition 14 (Strong normalisation for evaluation of typed expressions). *Given an expression e of type t , typed within the typing environment G ,*

$$\text{sn_e } G \ t \ e \stackrel{\text{def}}{=} \forall E. \text{envsn } (G \upharpoonright_{\text{fv}(e)}) \ E \Rightarrow \text{sn_exec } t \ E \ e$$

We use $G \upharpoonright_{\text{fv}(e)}$ instead of G to accommodate the restricted environment used in closures, as defined in Definition 8. This adjustment is necessary to re-establish Proposition 3. Further discussion on Proposition 3 and the modifications made to restore it can be found in Section 6.6.1.

If we only require $\text{envsn } G \ E$ for any E in the definition of sn_e , the inductive hypothesis for the function expression $\text{Fn } s \ e$ becomes inapplicable. This prevents us from proving that the evaluated closure value is strongly normalising with respect to the type of the function expression, specifically $\text{sn_v } (\text{fnT } t \ t') \ (\text{Clos } s \ e \ (E \upharpoonright_{\text{fv}(e)} \setminus s))$. This is because, according to Definition 11, the body e in the function closure must be evaluated in the restricted dynamic environment $E \upharpoonright_{\text{fv}(e)} \setminus s$ captured by the closure. Therefore, we need to instantiate the dynamic environment in the inductive hypothesis for e as $E \upharpoonright_{\text{fv}(e)} \setminus s$. However, this necessitates first proving $\text{envsn } G[s := t] \ (E \upharpoonright_{\text{fv}(e)} \setminus s)$ in order to apply the results of the inductive hypothesis for e . Unfortunately, $\text{envsn } G[s := t] \ (E \upharpoonright_{\text{fv}(e)} \setminus s)$ cannot be deduced from $\text{envsn } G[s := t] \ E$, which is the premise for the current goal. This is because G may contain arbitrary variables that are not included in $E \upharpoonright_{\text{fv}(e)} \setminus s$.

Thus, we modify the definition of sn_e to use a restricted typing environment that aligns with the restricted dynamic environment in a function closure. We require this restricted typing environment to include types for at most all free variables of the expression being evaluated. After introducing this restriction, the condition for the inductive hypothesis of e becomes

$$\text{envsn } (G \upharpoonright_{\text{fv}(e)})[s := t] \ (E \upharpoonright_{\text{fv}(e)})[s := v],$$

which can be deduced from $\text{envsn } (G \upharpoonright_{\text{fv}(e) \setminus \{s\}}) \ E$, the new premise of the current goal following the restriction of the typing environment.

5.2 Auxillary Lemmas

In this section, we present several lemmas related to the definitions introduced in Section 5.1. These lemmas will be utilised in the proof of the Fundamental Lemma.

Similar to the lemma of canonical forms for typed values (Lemma 5), we have a lemma of canonical forms for strongly normalising values. This lemma allows us to express a value v that satisfies $\text{sn_v } t \ v$ for some t in a concrete syntax. This is particularly useful when proving Lemma 12 and 13. Additionally, Lemma 10 is used to prove the **If**, **App**, and **Case** cases in the Fundamental Lemma proof, as it enables us to represent the boolean guard value, function closure value, and sum value in their concrete syntax rather than simply referring to them as v .

Lemma 10 (Canonical forms for strongly normalising values).

1. (Boolean)

$$\text{sn_v } \text{boolT } v \iff \exists b. v = \text{BoolV } b$$

2. (Pair)

$$\begin{aligned} \text{sn_v } (\text{pairT } t_1 \ t_2) \ v &\iff \\ \exists v_1 \ v_2. v &= \text{PairV } v_1 \ v_2 \wedge \text{sn_v } t_1 \ v_1 \wedge \text{sn_v } t_2 \ v_2 \end{aligned}$$

3. (Closure)

$$\begin{aligned}
& \text{sn_v} (\text{fnT } t_1 \ t_2) \ v \Rightarrow \\
& \quad \exists s \ e \ E. \\
& \quad v = \text{Clos } s \ e \ E \wedge \\
& \quad \forall v_0. \\
& \quad \text{sn_v } t_1 \ v_0 \Rightarrow \\
& \quad (\exists cl \ v'. \\
& \quad \quad \text{eval_exp } cl \ E[s := v_0] \ e = \text{Value } v' \wedge \text{sn_v } t_2 \ v') \vee \\
& \quad \exists cl \ \text{exn}. \text{eval_exp } cl \ E[s := v_0] \ e = \text{Exn } \text{exn}
\end{aligned}$$

4. (Sum)

$$\begin{aligned}
& \text{sn_v} (\text{sumT } t_1 \ t_2) \ v \Rightarrow \\
& \quad \exists v_0. v = \text{SumLV } v_0 \wedge \text{sn_v } t_1 \ v_0 \vee v = \text{SumRV } v_0 \wedge \text{sn_v } t_2 \ v_0
\end{aligned}$$

Proof: Immediately follows from Definition 11. $\square$

The following lemmas regarding $\text{envsn } G \ E$ prove to be quite useful in the proof of the Fundamental Lemma, particularly when we need to pair G with a larger or smaller E while maintaining the envsn relation.

Lemma 11. *Given typing environments G , G' , dynamic environments E , E' , a value v , and a type t , we have*

1.

$$\text{envsn } G \ E \wedge G' \sqsubseteq G \Rightarrow \text{envsn } G' \ E$$

2.

$$\text{envsn } G \ E \wedge \text{sn_v } t \ v \Rightarrow \text{envsn } G[s := t] \ E[s := v]$$

3.

$$\text{envsn } G \ E \wedge E \sqsubseteq E' \Rightarrow \text{envsn } G \ E'$$

4.

$$\text{envsn } G \ E \wedge E' \sqsubseteq E \wedge \text{domain}(G) \subseteq \text{domain}(E') \Rightarrow \text{envsn } G \ E'$$

Proof: Follows directly from Definition 12. $\square$

Property 1 enables us to establish $\text{envsn } G' \ E$ for some G' that is smaller than G . This is particularly useful when we have $\text{envsn } (G \upharpoonright_{\text{fv}(e)}) \ E$ and wish to deduce $\text{envsn } (G \upharpoonright_{\text{fv}(e_1)}) \ E$ and $\text{envsn } (G \upharpoonright_{\text{fv}(e_2)}) \ E$ in order to apply the inductive hypotheses for the sub-expressions e_1 and e_2 of e . This is because the free variables of an expression are typically defined as the union of the free variables of each of its sub-expressions. Property 2 is used in the **Let** and **Sum** cases of the Fundamental Lemma proof.

Property 3 allows us to establish the envsn property between G and a larger E . This lemma is used to re-establish the envsn property required by the inductive hypotheses when the variable x , bound by **Fn**, **Let**, or **Case**, is not utilised in a later expression e . As a result, the variable x is not included in the typing environment for e , as x is not a free variable of e . However,

our dynamic rules do not account for this information and still evaluate e with the previous environment, updated with the value of x . In this case, Property 1 is not directly applicable since we do not update the typing environment. Instead, with Property 3, we can extend the dynamic environment to include additional information about the values of unused variables.

Property 4 is used to address the restricted function closure in the **Fn** case of the Fundamental Lemma proof. The premise of our current goal provides us with $\text{envsn } (G \upharpoonright_{\text{fv}(e) \setminus \{s\}}) E$, but we need to establish the envsn property for the restricted environment in the function closure $\text{Clos } s \ e \ (E \upharpoonright_{\text{fv}(e) \setminus \{s\}})$. So we can apply the inductive hypothesis for the function body expression e stored in the closure. Property 4 enables us to reduce the dynamic environment, provided that the modified environment still encompasses all variables included in the typing environment.

Finally, we require the last two lemmas to address the **BinOp** and **Uop** cases. In the **BinOp** $\text{bop } e_1 \ e_2$ case, if the inductive hypotheses indicate that e_1 and e_2 both evaluate to values v_1 and v_2 , respectively, we need Lemma 12 to ensure that, given $\text{sn.v } t_1 \ v_1$ and $\text{sn.v } t_2 \ v_2$, the expression $\text{eval_bop } \text{bop } v_1 \ v_2$ always returns either a value or an exception. The same reasoning applies to the **Uop** case.

Lemma 12 (Strong normalisation for eval_bop). *Given values v_1, v_2 , types t_1, t_2 , and a binary operator bop , we have*

$$\begin{aligned} \text{sn.v } t_1 \ v_1 \wedge \text{sn.v } t_2 \ v_2 \wedge \text{boptype } \text{bop } t_1 \ t_2 \ ty &\Rightarrow \\ (\exists v. \text{eval_bop } \text{bop } v_1 \ v_2 = \text{Value } v \wedge \text{sn.v } ty \ v) \vee \\ \exists \text{exn}. \text{eval_bop } \text{bop } v_1 \ v_2 = \text{Exn } \text{exn} \end{aligned}$$

Proof: boptype is non-recursive. We prove by performing cases analyses based on boptype rules and each case follows from Lemma 10. $\square$

Lemma 13 (Strong normalisation for eval_uop). *Given a values v , a type t , and a unary operator uop , we have*

$$\begin{aligned} \text{sn.v } t_1 \ v \wedge \text{uoptype } \text{uop } t_1 \ ty &\Rightarrow \\ (\exists v'. \text{eval_uop } \text{uop } v = \text{Value } v' \wedge \text{sn.v } ty \ v') \vee \\ \exists e. \text{eval_uop } \text{uop } v = \text{Exn } e \end{aligned}$$

Proof: Same as the above. $\square$

5.3 The Fundamental Lemma

Using the definitions from Section 5.1, we state the Fundamental Lemma as:

Lemma 14 (The Fundamental Lemma). *Given an expression e of type t in typing environment G , we have*

$$G \vdash_s e : t \Rightarrow \text{sn.e } G \ t \ e$$

5.3.1 Strategy

We prove this using rule induction over Definition 5. The primary strategy in each case is to first derive the envsn property required by the inductive hypotheses from the premise of the current goal. Subsequently, we consider the sub-expressions of the current expression, which will either evaluate to a value or an exception based on the corresponding inductive hypotheses. If any sub-expression evaluates to an exception, the monadic binds employed in Definition 8 will ensure that the evaluation in the goal returns that exception.

We need to identify a clock value under which the first sub-expression that evaluates to an exception, along with all preceding sub-expressions that evaluate to values, continues to produce the same results. By applying Lemma 1 and 2, we can consistently use the summation of the clock values employed to evaluate these sub-expressions as the clock value for the evaluations in the goal statements, ensuring that they yield the same outcomes.

If no sub-expression evaluates to an exception, we either have an inductive hypothesis for the evaluation of some sub-expression that utilises the evaluated values, or we have an external `eval_bop` or `eval_uop` that makes use of those values. In the first case, we need to establish the `envsn` property required by the inductive hypothesis, which typically relies on Lemma 11. In the second case, we must apply Lemma 12 and 13.

The proof strategy itself is fairly straightforward; however, the evaluation of many expressions requires evaluating sub-expressions first (as outlined in Definition 8). This results in numerous similar cases, each requiring slightly different tactics, which contributes to the length of the HOL proof. For instance, the expression `BinOp bop e1 e2` requires that we first evaluate e_1 and e_2 before calling `eval_bop` with the evaluated values. This leads to four distinct cases, as both e_1 and e_2 can evaluate to either a value or an exception.

Additionally, as discussed in Lemma 11 (3), if the bound variable x is not used in a sub-expression e , the `envsn` statement in the inductive hypothesis for e will be `envsn (G |fv(e)) E[x := v]`. In contrast, if x is used in e , the statement would be `envsn (G |fv(e))[x := t] E[x := v]`. These two cases require slightly different proofs for reasons discussed in Section 5.2, and this added layer of distinction increases the number of generated cases, particularly for the `Fn`, `Let`, and `Case` scenarios.

5.3.2 Proof

Base cases. Evaluation for literals always return values and those values are related to the types of literal expressions by `sn_v`. For variable expressions, the goal follows from Definition 12.

Function. Consider a function expression `Fn s e` of type `fnT t1 t2`. Since we are using induction over type checking relation (Definition 5), we have one inductive hypothesis for evaluating e . According to Definition 8, a function expression always evaluates to a closure. We then need to show this closure is related to `fnT t1 t2` by `sn_v`. According to Definition 11, we need to demonstrate that for any v such that `sn_v t1 v`, evaluating e in the closure's environment extended with binding s to v either returns a value or an exception. This aligns with the result of the inductive hypothesis for e . From the discussion in Section 5.1, we know that the `envsn` property required to use the inductive hypothesis for e can be deduced from the premise of the current goal.

If expression. Consider an if expression `If bg e1 e2`. The `If` case follows the strategy outlined in Section 5.3.1. According to Definition 14, the condition for `sn_exec t E (If bg e1 e2)` requires `envsn (G |fv(If bg e1 e2)) E`. Since the free variables of `If bg e1 e2` are the union of those in bg , e_1 , and e_2 , Lemma 11 (1) allows us to establish `envsn (G |fv(bg)) E`, `envsn (G |fv(e1)) E`, and `envsn (G |fv(e2)) E`. With these, we can apply the inductive hypotheses for bg , e_1 , and e_2 .

Since each of them can evaluate to either a value or an exception, this generates 2^3 cases. If bg evaluates to an exception, then `If bg e1 e2` will evaluate to an exception regardless of the outcomes of e_1 and e_2 . If bg evaluates to a value v , we apply the canonical form lemma (Lemma 10) to express v as `BoolV b`. Depending on the value of b and the evaluation results of e_1 and e_2 (which

could be either a value or an exception as given by the corresponding inductive hypothesis), we use Lemma 1 and 2, instantiated with the clock value $c_{bg} + c_1 + c_2$, to prove the goal.

Binary operation. Consider a binary operation $\text{BinOp } bop \ e_1 \ e_2$. The BinOp case is similar to the If case and follows the strategy outlined in Section 5.3.1. From the premise of the current goal, we can deduce the necessary envsn conditions to use the inductive hypotheses for e_1 and e_2 by applying Lemma 11 (1). Since both e_1 and e_2 can evaluate to either a value or an exception, this generates 2^2 cases.

If either e_1 or e_2 evaluates to an exception, we use Lemma 1 and 2 with the clock value $c_1 + c_2$ to prove the goal. If both e_1 and e_2 evaluate to values v_1 and v_2 respectively, the inductive hypotheses provide $\text{sn}_v \ t_1 \ v_1$ and $\text{sn}_v \ t_2 \ v_2$. We then instantiate Lemma 1 with a clock value of $c_1 + c_2$ so that e_1 and e_2 are evaluated with the same clock value, reaching the same results v_1 and v_2 . Finally, Lemma 12 guarantees that $\text{eval_bop } v_1 \ v_2$ will return either a value or an exception, completing the proof.

Unary operation. Same as the binary operation case.

Let expression. Consider a Let expression $\text{Let } s \ e_1 \ e_2$. Similar to the previous cases, we use Lemma 11 (1) to deduce the envsn conditions required for applying the inductive hypothesis of e_1 . If e_1 evaluates to an exception, the proof is complete. If e_1 evaluates to a value v , we need to show that evaluating e_2 within $E[s := v]$ will either return a value or an exception. This outcome aligns with the inductive hypothesis for e_2 .

By applying Lemma 11 (2), we establish $\text{envsn } G[s := t_1] \ E[s := v]$. This allows us to use the inductive hypothesis for e_2 , confirming that evaluating e_2 within $E[s := v]$ will result in either a value or an exception. Finally, by instantiating Lemma 1 and 2 with a clock value of $c_1 + c_2$, we complete the proof.

As discussed in Section 5.2 regarding Lemma 11 (3), if s is not used in e_2 , meaning s is not a free variable in e_2 , the inductive hypothesis for e_2 requires $\text{envsn } G \ E[s := v]$. Lemma 11 (3) allows us to deduce $\text{envsn } G \ E[s := v]$ in this situation.

Application. Application case follows the strategy stated in Section 5.3.1 and is similar to the If case.

Case expression. Consider a case expression $\text{Case } e \ s_1 \ e_1 \ s_2 \ e_2$. The proof follows the strategy outlined in Section 5.3.1, where Lemma 11 (1) is used to apply the inductive hypothesis for e . However, additional case analyses are needed to determine whether s_1 is a free variable in e_1 and whether s_2 is a free variable in e_2 . This additional analysis is similar to what is required in the Let case, where either Lemma 11 (3) or Lemma 11 (2) is used to apply the inductive hypothesis for each situation. $\square$

5.3.3 Strong Normalisation

We now state the strong normalisation property for **Sprinkles**:

Corollary 3 (Strong normalisation for **Sprinkles**). *Given a closed expression e of type t , we have*

$$\begin{aligned} \emptyset \vdash_s e : t \Rightarrow \\ (\exists cl \ v \ E. \text{eval_exp } cl \ E \ e = \text{Value } v \wedge \text{sn}_v \ t \ v) \vee \\ \exists cl \ exn \ E. \text{eval_exp } cl \ E \ e = \text{Exn } exn \end{aligned}$$

Proof: Follows from Lemma 14 $\square$

Corollary 3 says that if a closed expression of **Sprinkles** is well-typed, then it either evaluates to a value or to an exception.

This result is nice on its own; however, Corollary 3 only establishes the existence of such an environment E . As a result, this finding is not directly applicable in the progress and type preservation proof for the enriched choreography. Instead, we aim to show that the evaluation of a typed expression terminates in the localised choreography environment. Theorem 3 in Section 6.4.2 provides this crucial result.

Kalas Enriched with Sprinkles

We introduce the enriched Kalas, where local computations are handled by **Sprinkles**. As a result, the external function $f(v)$ in the previous **Let** rule of Kalas (Definition 1) is replaced with a **Sprinkles** expression e . Messages in Kalas used to be a list of strings, but now they are string values in **Sprinkles**. Since evaluations in **Sprinkles** may return an exception, we introduce new transition rules in the enriched Kalas to handle local computation failure.

We provide typing rules for the enriched Kalas. Our choreography does not return values and thus our rules only specify whether a choreography type checks. Kalas uses *states* to store values for variables located at a certain process, so we introduce a similar *typing state* that stores types for variables located at a certain process.

Since a local expression e cares neither about values of variables in other processes nor about their types, we introduce the concept of *localised* environment and *localised* typing environment to describe the result of filtering out values or types of variables in other processes. So e can be evaluated in the *localised* environment and typed in the *localised* typing environment.

We prove that our enriched Kalas has an important property: *progress*. The strong normalisation property of **Sprinkles** is crucial for the progress proof. We also prove that the non-recursive, synchronous transitions in the enriched Kalas has type preservation property, where the type soundness and strong normalisation of local computations is required.

6.1 Syntax

Definition 15 (Syntax for Kalas enriched with **Sprinkles**). *Choreographies in Kalas enriched with Sprinkles, ranged over by C , are inductively defined by the grammar*

$$\begin{array}{llll}
 C ::= & p_1.v_1 \Rightarrow p_2.v_2; C & (com) & p_1 \Rightarrow p_2[b]; C & (sel) \\
 & \text{if } v@p \text{ then } C_1 \text{ else } C_2 & (if) & \text{Let } v \text{ } p \text{ } e \text{ } C & (let) \\
 & \mu X. C & (fix) & X & (var) \\
 & \mathbf{0} & (nil) & &
 \end{array}$$

In $\text{Let } v \text{ } p \text{ } e \text{ } C$, e represents a **Sprinkles** expression and (v, p) represents a variable v located at process p . The message values are **Sprinkles**'s results (Definition 4.1). Other syntax are the same

as Kalas' (Definition 1).

Free variables in a choreography are defined in Definition 16. $\mathbf{0}$ does not contain any free variables. **SEL** and **FIX** do not introduce any new free variables. In **if** $v@p$ **then** c_1 **else** c_2 , the pair (v, p) becomes free. In $p_1.v_1 \rightarrow p_2.v_2; c$, the variable (v_1, p_1) is free, while (v_2, p_2) becomes bound. The free variables in **Let** $v \ p \ e \ c$ are defined as the union of all free variables in the expression e located at p and the free variables in the sequencing choreography c , excluding (v, p) .

For $\mathbf{0}$, **IfThen**, **SEL**, and **FIX**, the set of free variables in the current choreography is a superset of the free variables in the subsequent choreography. However, this does not necessarily hold for **Com** and **Let**. Therefore, as a closed choreography progresses, it may become open.

Definition 16 (Free variables of choreography).

$$\begin{aligned} \text{fv}(\mathbf{if} \ v@p \ \mathbf{then} \ c_1 \ \mathbf{else} \ c_2) &\stackrel{\text{def}}{=} \{ (v, p) \} \cup (\text{fv}(c_1) \cup \text{fv}(c_2)) \\ \text{fv}(p_1.v_1 \rightarrow p_2.v_2; c) &\stackrel{\text{def}}{=} \{ (v_1, p_1) \} \cup (\text{fv}(c) \setminus \{(v_2, p_2)\}) \\ \text{fv}(\mathbf{Let} \ v \ p \ e \ c) &\stackrel{\text{def}}{=} \{ (s, p) \mid s \in \text{fv}(e) \} \cup (\text{fv}(c) \setminus \{(v, p)\}) \\ \text{fv}(p \rightarrow q[b]; c) &\stackrel{\text{def}}{=} \text{fv}(c) \\ \text{fv}(\mu X. c) &\stackrel{\text{def}}{=} \text{fv}(c) \end{aligned}$$

6.2 Statics

6.2.1 Syntax and Typing Rules

A typing state, Γ , is a finite map that associates pairs (v, p) with types from **Sprinkles**'s type system (see Definition 4). A type statement in the enriched Kalas is expressed as $\Gamma, \Theta \vdash c \checkmark$, where Γ is the typing state, Θ is a set of processes, and c is the choreography being typed.

We also introduce the *localised* typing environment, $\text{localise } \Gamma \ p$, for typing **Sprinkles** expressions. Since a **Sprinkles** expression e , located at a process p , only depends on variables at p , we filter out variables associated with other processes and retain only the types of variables located at p in $\text{localise } \Gamma \ p$. This procedure is formally defined in Definition 17.

Definition 17 (Localised environments). *Given a state or a typing state s and a process p , we define the localise environment as a function composition*

$$\text{localise } s \ p \stackrel{\text{def}}{=} s \circ (\lambda vn. (vn, p))$$

Definition 18 (Typing rules for Kalas enriched with Sprinkles).

$$\begin{array}{c}
\text{CT-COM} \frac{\Gamma(v_1, p_1) = \text{strT} \quad \{p_1; p_2\} \subseteq \Theta \quad p_1 \neq p_2 \quad \Gamma[(v_2, p_2) := \text{strT}], \Theta \vdash c \checkmark}{\Gamma, \Theta \vdash p_1.v_1 \Rightarrow p_2.v_2; c \checkmark} \\
\\
\text{CT-IF} \frac{\Gamma(v, p) = \text{boolT} \quad \Gamma, \Theta \vdash c_1 \checkmark \quad \Gamma, \Theta \vdash c_2 \checkmark}{\Gamma, \Theta \vdash \text{if } v@p \text{ then } c_1 \text{ else } c_2 \checkmark} \\
\\
\text{CT-LET} \frac{\text{localise } \Gamma \ p \vdash_s e : \text{ety} \quad \Gamma[(v, p) := \text{ety}], \Theta \vdash c \checkmark}{\Gamma, \Theta \vdash \text{Let } v \ p \ e \ c \checkmark} \\
\\
\text{CT-SEL} \frac{\{p_1; p_2\} \subseteq \Theta \quad p_1 \neq p_2 \quad \Gamma, \Theta \vdash c \checkmark}{\Gamma, \Theta \vdash p_1 \Rightarrow p_2[b]; c \checkmark} \quad \text{CT-NIL} \frac{}{\Gamma, \Theta \vdash \mathbf{0} \checkmark} \\
\\
\text{CT-FIX} \frac{\Gamma, \Theta \cup \{dn\} \vdash c \checkmark}{\Gamma, \Theta \vdash \mu dn. c \checkmark} \quad \text{CT-CALL} \frac{\{dn\} \subseteq \Theta}{\Gamma, \Theta \vdash dn \checkmark}
\end{array}$$

The typing rules for the enriched Kalas are presented in Definition 18. $\mathbf{0}$ always type-checks, regardless of the context provided by Γ and Θ . For $p_1.v_1 \Rightarrow p_2.v_2; c$, the sender p_1 's variable v_1 must have the type strT , and the subsequent choreography c must be type-checked under the condition that the receiver p_2 's variable v_2 holds a value of type strT . Note that self-communication is not permitted. In the case of **Let**, the **Sprinkles** expression e must be typed within a localised typing environment at the process p . The result of the local computation from e will be stored in the variable v at p . Therefore, the choreography c needs to type-check given the type of v at p .

Since local computations in the enriched Kalas assign values to free variables by looking them up in the localised environment, we define $\text{chorEnvtype } \Gamma \ s$ to ensure that if a choreography is typed by Γ , then for any process p , we have $\text{envtype}(\text{localise } \Gamma \ p)(\text{localise } s \ p)$. Combined with the **Let** typing rule in Definition 18, this guarantees type soundness for local computations. Additionally, $\text{chorEnvtype } \Gamma \ s$ ensures that the state s contains values for all free variables in the choreography typed under Γ .

Definition 19 (Typing constrains for localised environments). *Given a typing state Γ and a state s , $\text{chorEnvtype } \Gamma \ s$ is defined as*

$$\begin{aligned}
\text{chorEnvtype } \Gamma \ s &\stackrel{\text{def}}{=} \\
&\forall p. \text{envtype}(\text{localise } \Gamma \ p)(\text{localise } s \ p)
\end{aligned}$$

6.2.2 Typing Examples

The choreography in Example 1.0.2 can be typed using the rules from Definition 18, given an initial typing state of $\Gamma = (\text{var}, \text{server}) : \text{strT}$ and a set of processes $\Theta = \text{server}, \text{client}$.

If process *server* stores a non-string value in *var*, the choreography cannot be typed anymore since the condition $\Gamma(\text{var}, \text{server}) = \text{strT}$ would not be satisfied.

6.3 Dynamics

Our transition rules are similar to those of Kalas, with a few modifications. The CS-COM rule requires that (v_1, p_1) holds a **Sprinkles** value of type **StrV**. If this condition is not met, a communication exception is triggered, and the choreography transitions to **0**.

According to the CS-LET rule, if the expression e at process p evaluates to a value v within the localised environment at p , and if the localised environment contains bindings for all free variables in e , then the state s updates to store the value v in variable v at p , and the choreography proceeds to c . The additional condition, $\text{fv}(e) \subseteq \text{domain}(\text{localise } s \ p)$, is necessary to maintain Proposition 3, as discussed in Section 6.6. The outcome of the local evaluation is recorded in the transition label.

The C-LETENX rule is similar to C-LET; if a local evaluation fails, the choreography transitions to **0**. In C-IFTRUE and C-IFFALSE, we rely on **Sprinkles**'s boolean values, and the rules are straightforward. If the value of variable v at p is not a boolean, a choreography failure occurs, causing a transition to **0**.

Definition 20 (Transition rules for Kalas enriched with Sprinkles).

$$\begin{array}{c}
\text{CS-COM} \frac{s(v_1, p_1) = \text{StrV } d \quad p_1 \neq p_2}{s \triangleright p_1.v_1 \rightarrow p_2.v_2; c \xrightarrow[\epsilon]{p_1.v_1 \triangleright p_2.v_2} s[(v_2, p_2) := \text{StrV } d] \triangleright c} \\
\text{CS-COMEXN} \frac{s(v_1, p_1) = v \quad \forall s. v \neq \text{StrV } s}{s \triangleright p_1.v_1 \rightarrow p_2.v_2; c \xrightarrow[\epsilon]{\text{LComExn } p_1 \ v_1 \ p_2 \ v_2} s \triangleright \mathbf{0}} \\
\text{CS-LET} \frac{\text{eval_exp } cl(\text{localise } s \ p) \ e = \text{Value } ev \quad \text{fv}(e) \subseteq \text{domain}(\text{localise } s \ p)}{s \triangleright \text{Let } v \ p \ e \ c \xrightarrow[\epsilon]{\text{let } v@p, (\text{Value } ev)} s[(v, p) := ev] \triangleright c} \\
\text{CS-LETENX} \frac{\text{eval_exp } cl(\text{localise } s \ p) \ e = \text{Exn } exn \quad \text{fv}(e) \subseteq \text{domain}(\text{localise } s \ p)}{s \triangleright \text{Let } v \ p \ e \ c \xrightarrow[\epsilon]{\text{let } v@p, (\text{Exn } exn)} s \triangleright \mathbf{0}} \\
\text{CS-IFTRUE} \frac{s(v, p) = \text{BoolV } \mathbf{T}}{s \triangleright \text{if } v@p \text{ then } c_1 \text{ else } c_2 \xrightarrow[\epsilon]{\tau_p} s \triangleright c_1} \\
\text{CS-IFFALSE} \frac{s(v, p) = \text{BoolV } \mathbf{F}}{s \triangleright \text{if } v@p \text{ then } c_1 \text{ else } c_2 \xrightarrow[\epsilon]{\tau_p} s \triangleright c_2} \\
\text{CS-IFEXN} \frac{s(v, p) = w \quad \neg \text{is_BoolV } w}{s \triangleright \text{if } v@p \text{ then } c_1 \text{ else } c_2 \xrightarrow[\epsilon]{\tau_p} s \triangleright \mathbf{0}}
\end{array}$$

6.4 Progress

In this section, we demonstrate that a typed choreography can always make progress, proving that the enriched Kalas with **Sprinkles** is deadlock-free. However, since a closed choreography can transition to an open one, we formalise our progress statement to cover general choreographies.

To ensure that the state s includes values for all free variables in a typed choreography, we use $\text{chorEnvtype } \Gamma \ s$. The concept $\text{chorEnvsn } \Gamma \ s$ is defined similarly to chorEnvtype , but it ensures that

s contains values that are strongly normalising, adhering to the typing constraints specified in Γ . These constraints play a crucial role in proving the progress theorem.

We also establish that the evaluation of typed **Sprinkles** expressions within a localised environment always results in either a value or an exception. This outcome is guaranteed by the strong normalisation property of **Sprinkles**, which is essential for the progress proof.

6.4.1 Formalisation

First, we define $\text{chorEnvsn } \Gamma \ s$, which stipulates that the state s must contain strongly normalising values for all free variables in the choreography typed under Γ .

Definition 21 (Strong normalisation constrains for localised environments). *Given a typing state Γ and a state s , $\text{chorEnvsn } \Gamma \ s$ is defined as*

$$\text{chorEnvsn } \Gamma \ s \stackrel{\text{def}}{=} \forall p. \text{envsn } (\text{localise } \Gamma \ p) (\text{localise } s \ p)$$

We state the progress property for Kalas enriched with **Sprinkles** as the following:

Theorem 2 (Progress). *Given a typing state Γ , a set of processes Θ , an initial state s , and a choreography c that is not $\mathbf{0}$, we state the progress property as*

$$\begin{aligned} \Gamma, \Theta \vdash c \checkmark \wedge \text{chorEnvtype } \Gamma \ s \wedge \text{chorEnvsn } \Gamma \ s \Rightarrow \\ \exists \tau \ l \ s' \ c'. s \triangleright c \xrightarrow[l]{\tau} s' \triangleright c' \vee \neg \text{not_finish } c \end{aligned}$$

It states that for a choreography c typed under the typing state Γ , if the corresponding state s contains variables typed in Γ with values that are both of the correct types and strongly normalising, then c can either advance from state s or be equal to $\mathbf{0}$.

One might expect that any closed choreography that can be typed will always make progress, in line with the progress theorem. However, we know that a closed choreography may transition into an open choreography, as discussed in Section 6.1. This nuance diminishes the interest of such a statement. Furthermore, according to the transition rules of the enriched Kalas (Definition 20), the state s may be extended after a transition, meaning that a closed choreography starting from an empty state could end up in a non-empty state. Consequently, we frame the progress theorem for more general choreographies, which requires the addition of the constraints chorEnvtype and chorEnvsn .

6.4.2 Lemmas

We introduce several lemmas that are useful for the progress proof. The first lemma is the generation lemma, which enables us to derive types and typing information for variables, processes, and choreographies from a typed choreography.

Lemma 15 (Generation lemma). *Given a typed choreography, we have*

1. (Let)

$$\begin{aligned} \Gamma, \Theta \vdash \text{Let } v \ p \ e \ c \checkmark &\iff \\ \exists \text{ety}. \text{localise } \Gamma \ p \vdash_s e : \text{ety} \wedge \Gamma[(v, p) := \text{ety}], \Theta \vdash c \checkmark \end{aligned}$$

2. (If)

$$\begin{aligned} \Gamma, \Theta \vdash \text{if } v @ p \text{ then } c_1 \text{ else } c_2 \checkmark &\iff \\ \Gamma(v, p) = \text{boolT} \wedge \Gamma, \Theta \vdash c_1 \checkmark \wedge \Gamma, \Theta \vdash c_2 \checkmark \end{aligned}$$

3. (Com)

$$\begin{aligned} \Gamma, \Theta \vdash p_1.v_1 \rightarrow p_2.v_2; c \checkmark &\iff \\ \Gamma(v_1, p_1) = \text{strT} \wedge \{p_1; p_2\} \subseteq \Theta \wedge p_1 \neq p_2 \wedge \\ \Gamma[(v_2, p_2) := \text{strT}], \Theta \vdash c \checkmark \end{aligned}$$

4. (Sel)

$$\begin{aligned} \Gamma, \Theta \vdash p_1 \rightarrow p_2[b]; c \checkmark &\iff \\ \{p_1; p_2\} \subseteq \Theta \wedge p_1 \neq p_2 \wedge \Gamma, \Theta \vdash c \checkmark \end{aligned}$$

5. (Fix)

$$\Gamma, \Theta \vdash \mu dn. c \checkmark \iff \Gamma, \Theta \cup \{dn\} \vdash c \checkmark$$

Proof: Immediately follows from Definition 18. $\square$

The *Let* $v\ p\ e\ c$ case of the generation lemma is required by the *Let* case in the progress proof. Lemma 15 gives us that the local expression e is typed if the choreography is typed. This allows us to apply the strong normalisation theorem to e . Lemma 15 is also used in the type preservation proof.

The following lemma states that if a choreography c is typed under some Γ , then it does not exhibit any self-communication behaviours. Furthermore, if we have $\text{chorEnvtype } \Gamma\ s$, then the state s contains correctly typed values for all free variables in c .

Lemma 16 (Well-formedness). *Given a typing state Γ , a set of processes Θ , a state s , we have the following properties for a typed choreography c*

1. $\Gamma, \Theta \vdash c \checkmark \Rightarrow \text{no_self_communication } c$
2. $\Gamma, \Theta \vdash c \checkmark \wedge \text{chorEnvtype } \Gamma\ s \Rightarrow \text{fv}(c) \subseteq \text{domain}(s)$

Proof: By rule induction over Definition 18. $\square$

Strong Normalisation in the Localised Environments

As discussed in Section 5.3.3, Corollary 3 shows the strong normalisation property of the Sprinkles evaluation function. However, it cannot be applied in the progress or type preservation proofs because we need to evaluate expressions in the localised environment, and Corollary 3 does not guarantee that evaluation in a localised environment will terminate. Therefore, we require Theorem 3, which states that if an expression e is typed in a typing environment localised at p , and if we have $\text{chorEnvsn } \Gamma\ s$, then evaluating e in the dynamic environment localised at p will either return a value or raise an exception.

Theorem 3 (Strong normalisation in localised environmet).

$$\begin{aligned} \text{chorEnvsn } \Gamma\ s \wedge \text{localise } \Gamma\ p \vdash_s e : t &\Rightarrow \\ (\exists cl\ v. \text{eval_exp } cl\ (\text{localise } s\ p)\ e = \text{Value } v \wedge \text{sn_v } t\ v) \vee \\ \exists cl\ exn. \text{eval_exp } cl\ (\text{localise } s\ p)\ e = \text{Exn } exn \end{aligned}$$

Proof: Follows from the Fundamental Lemma (Lemma 14). $\square$

6.4.3 Proof

We prove Theorem 2 through case analysis of the choreography syntax. From $\Gamma, \Theta \vdash c \checkmark$ and $\text{chorEnvtype } \Gamma s$, we know that c does not contain any self-communications, and that state s contains correctly typed values for all free variables in c , as established in Lemma 16. The cases for Sel and Fix follow directly from this result.

In the case of **if** $v@p$ **then** c_1 **else** c_2 , we know that s stores a value for the variable v at process p . This value can take one of three forms: BoolV T , BoolV F , or any other type, corresponding to CS-IFTRUE, CS-IFFALSE, and CS-IFEXN, respectively. A similar analysis applies to the case of Com.

In the case of **Let** $v p e c$, Lemma 15 tells us that e is typed in $\text{localise } \Gamma p$. This allows us to apply Theorem 3, which states that e will either evaluate to a value or produce an exception. As discussed in Section 6.3, to apply SC-LET or SC-LETEXN, we must show that $\text{fv}(e) \subseteq \text{domain}(\text{localise } \Gamma p)$. Lemma 7 indicates that the localised typing environment $\text{localise } \Gamma p$ contains types for the free variables of e . Given $\text{chorEnvtype } \Gamma s$ and Definition 19, we know that the domain of Γ is a subset of the domain of s . Therefore, SC-LET or SC-LETEXN can be applied, solving the goal. $\square$

6.5 Type Preservation

We formalise the type preservation theorem as:

Theorem 4 (Type preservation). *Given a typing state Γ , a set of processes Θ , and an initial state s ,*

$$\begin{aligned} & \text{not.finish } c \wedge \text{chorEnvsn } \Gamma s \wedge \text{chorEnvtype } \Gamma s \wedge \\ & \Gamma, \Theta \vdash c \checkmark \wedge s \triangleright c \xrightarrow[l]{\tau} s' \triangleright c' \Rightarrow \\ & \exists \Gamma'. \\ & \text{chorEnvsn } \Gamma' s' \wedge \text{chorEnvtype } \Gamma' s' \wedge \Gamma', \Theta \vdash c' \checkmark \end{aligned}$$

Theorem 4 states that if a choreography is typed under Γ and state s contains strongly normalising values of the correct types according to Γ , then when the choreography transitions to the next state, there exists a typing state Γ' such that the new choreography is typed and the new state s' contains strongly normalising values of the correct types according to Γ' .

We cannot guarantee that the new choreography will still be typed in the same typing state, as advancing the choreography may introduce new free variables. This is the same reason we formulated the progress theorem for a more general choreography rather than a closed one, as discussed in Section 6.4.1.

Proof: We prove Theorem 4 by induction on the transition relation defined in Definition 20. The cases for 0, Sel, and If follow directly from Definitions 18 and 20.

To prove the cases for Com and Let, we need the following lemma about chorEnvtype and chorEnvsn :

Lemma 17 (Updating states). *Given a typing state Γ , a state s , and some value v of type t ,*

1. $\text{chorEnvtype } \Gamma s \wedge \vdash_s v : t \Rightarrow$
 $\forall vn p. \text{chorEnvtype } \Gamma[(vn, p) := t] s[(vn, p) := v]$
2. $\text{chorEnvsn } \Gamma s \wedge \text{sn}_v t v \Rightarrow$
 $\forall vn p. \text{chorEnvsn } \Gamma[(vn, p) := t] s[(vn, p) := v]$

Proof: Follows from Definition 19 and 21. $\square$

In the case of $p_1.v_1 \Rightarrow p_2.v_2; c$, we know that CS-COM must apply since the choreography type checks, indicating that the variable v_1 at p maps to a string value $\text{StrV } d$. By Lemma 15, we have $\Gamma[(v_2, p_2) := \text{strT}], \Theta \vdash c \checkmark$. Furthermore, by Lemma 17, we obtain

$$\text{chorEnvtype } \Gamma[(v_2, p_2) := \text{strT}] s[(v_2, p_2) := \text{StrV } d]$$

and $\text{chorEnvsn } \Gamma[(v_2, p_2) := \text{strT}] s[(v_2, p_2) := \text{StrV } d]$, which match the goal.

In the case of **Let** $x \ p \ e \ c$, if e evaluates to an exception, the proof is straightforward, as we transition into $\mathbf{0}$ when an exception occurs, and $\mathbf{0}$ can be typed by any Γ and Θ . When e evaluates to a value v , Lemma 15 tells us that $\text{localise } \Gamma \ p \vdash_s e : t$ holds, along with $\Gamma[(x, p) := t], \Theta \vdash c \checkmark$. If we can show that $\vdash_s v : t$ and $\text{sn.v } t \ v$, then applying Lemma 17 will complete the proof.

Since e is typed in the localised typing environment $\text{localise } \Gamma \ p$, Theorem 3 and the type soundness of Sprinkles (Theorem 1) guarantee that e evaluates to either a value v' or an exception. In the case of an exception, as discussed above, the choreography transitions to $\mathbf{0}$, making the proof straightforward. For the former scenario, we have $\vdash_s v' : t$ and $\text{sn.v } t \ v'$. Next, we need to show that $v = v'$, which holds true since the evaluation function for Sprinkles is deterministic.

We provide proof sketches for the parallel transitions (rules can be found in Appendix A) even though we did not implement the proofs for parallel transitions in HOL4.

For the case of **If**, consider a choreography **if** $v@p$ **then** c_1 **else** c_2 . We know the choreography type checks, and by Lemma 15, we have $\Gamma(v, p) = \text{boolT}, \Gamma, \Theta \vdash c_1 \checkmark$, and $\Gamma, \Theta \vdash c_2 \checkmark$.

By the inductive hypotheses for c_1 and c_2 , we obtain $\Gamma_1, \Theta \vdash c'_1 \checkmark$ and $\Gamma_2, \Theta \vdash c'_2 \checkmark$. Additionally, we have $\text{chorEnvsn } \Gamma_1 \ s', \text{chorEnvsn } \Gamma_2 \ s', \text{chorEnvtype } \Gamma_1 \ s'$, and $\text{chorEnvtype } \Gamma_2 \ s'$.

By Definitions 21 and 19, we know that Γ_1 and Γ_2 assign the same type to variables in the intersection of their domains. Therefore, we can construct a Γ' that includes all mappings from Γ_1 and Γ_2 without encountering any contradictions. Consequently, we have $\text{chorEnvsn } \Gamma' \ s'$ and $\text{chorEnvtype } \Gamma' \ s'$.

By IF-SWAP, we can ensure that the parallel transitions at c_1 and c_2 do not involve the variable v at p . Thus, $(s(v, p)) = (s'(v, p))$. As a result, **if** $v@p$ **then** c'_1 **else** c'_2 can be typed by $\Gamma'[(v, p) := \text{boolT}]$. Similar analyses apply to the other parallel transition rules. $\square$

However, we do not provide a proof for type preservation in the case of **Fix**, as our typing system cannot type an unfolding choreography. Additionally, we do not address the transitions involving asynchronous behaviours due to time constraints in this project.

6.6 Discussions on Kalas Properties

6.6.1 Transitions in Larger States

The Kalas repository reveals an interesting property: If a larger state $z2$ contains $z1$, the choreography c will still transition into c' , with z' containing $z1'$.

Proposition 3 (Transitions in a larger state). *Given a state $z1$ and a larger state $z2$, we have*

$$z1 \triangleright c \xrightarrow[\tau]{\alpha} z'_1 \triangleright c' \wedge z1 \sqsubseteq z2 \Rightarrow \exists z'. z2 \triangleright c \xrightarrow[\tau]{\alpha} z' \triangleright c' \wedge z'_1 \sqsubseteq z'$$

Re-establishing this property requires two modifications.

Restricted Function Closure

The first modification involves changing the evaluation rule for function expressions in *Sprinkles*' (Definition 8) as mentioned in Section 4.3.1. This adjustment stipulates that a restricted environment, $\text{Clos } s \ e \ (E \upharpoonright_{\text{fv}(e)} \setminus s)$, is stored within the evaluated closure. This change is crucial because evaluating $\text{Fn } s \ E$ into $\text{Clos } s \ e \ E$ could invalidate Proposition 3. A larger state might yield a closure value that contains a larger environment, preventing the choreography from transitioning to the same c' when given z' .

This explains why it is beneficial to restrict E to include only bindings for the free variables of e , as the set of free variables in e remains unchanged, even when we are given a larger state.

We also argue that it is essential to exclude s from the environment. If we only use $\text{Clos } s \ e \ (E \upharpoonright_{\text{fv}(e)})$, the state z may lack a binding for s when evaluating $\text{Fn } s \ E$. Consequently, $\text{localise } z \ p \upharpoonright_{\text{fv}(e)}$ may not contain a binding for s when $s \in \text{fv}(e)$. In contrast, $\text{localise } z' \ p \upharpoonright_{\text{fv}(e)}$ may contain a binding for s , as z' is larger. This means that $\text{localise } z' \ p \upharpoonright_{\text{fv}(e)}$ may be larger than $\text{localise } z \ p \upharpoonright_{\text{fv}(e)}$.

Another potential approach to re-establish Proposition 3 is to define an equivalence relation on function closures. In this framework, closures with the same parameter and function body are considered equivalent if their environments contain identical bindings for the free variables in the function body, while excluding any bindings for the function parameter itself. However, this method introduces additional complexity, as it requires addressing choreographies that contain different yet equivalent function closures.

Using a restricted function closure requires employing a minimal typing environment (Corollary 2) in the Fn case to ensure type soundness (as discussed in Section 4.4.2). Additionally, this restricted closure environment requires the use of a limited typing environment, $G \upharpoonright_{\text{fv}(e)}$, in the definition of sn.e (Definition 14). If we opt for $\text{Clos } s \ e \ E$ instead, we can forgo both Corollary 2 and the modifications to Definition 14. However, this alternative would invalidate Proposition 3 by the reasons discussed above.

Modified Transition Rules for the Enriched Kalas

We introduce an additional constraint, $\text{fv}(e) \subseteq \text{domain}(\text{localise } z \ p)$, to the CS-LET and CS-LETENX rules for the enriched Kalas (as defined in Definition 20 in Section 6.3) to re-establish Proposition 3. In the case of CS-LET, we can evaluate a function expression $\text{Fn } s \ e$ into a closure value, regardless of the localised environment provided. This means that $\text{localise } z' \ p \upharpoonright_{\text{fv}(e)} \setminus s$ can still be larger than $\text{localise } z \ p \upharpoonright_{\text{fv}(e)} \setminus s$. Specifically, when $\text{localise } z \ p$ does not include all the free variables of e , $\text{localise } z' \ p$ may contain additional bindings for these free variables that are absent in $\text{localise } z \ p$.

However, this modification does not require any extra results to prove the Let transitions in the progress and type soundness proofs. This indicates that if we incorporate typing into the statement of Proposition 3, the changes made to the C-LET and C-LETENX rules can be omitted.

6.6.2 Synchronous Transitions

We have fixed the proof for Proposition 4 in Kalas' repository, which is one of the Kalas properties proved in the original development. The proposition states that if the state s contains values for all free variables of c , and c does not involve any self-communications, then for any label τ , the choreography can always advance synchronously until that label τ is consumed by a transition. If this does not occur, the choreography will advance to an end state, represented as $\mathbf{0}$.

Definition 22 (syncTrm).

1.

$$\begin{aligned} \text{syncTrm } k \ (s, p_1.v_1 \rightarrow p_2.v_2; c) \ \tau &\stackrel{\text{def}}{=} \\ &\text{if } k = 0 \text{ then None} \\ &\text{else if } \text{chor_match } \tau \ (p_1.v_1 \rightarrow p_2.v_2; c) \text{ then} \\ &\quad \text{Some } (\text{chor_tl } s \ (p_1.v_1 \rightarrow p_2.v_2; c)) \\ &\text{else} \\ &\quad \text{case some } str. \ s \ (v_1, p_1) = \text{StrV } str \text{ of} \\ &\quad \quad \text{None} \Rightarrow \text{None} \\ &\quad \quad | \text{ Some } str \Rightarrow \text{syncTrm } (k - 1) \ (s[(v_2, p_2) := \text{StrV } str], c) \ \tau \end{aligned}$$

2.

$$\begin{aligned} \text{syncTrm } k \ (s, \text{Let } v \ p \ e \ c) \ \tau &\stackrel{\text{def}}{=} \\ &\text{if } k = 0 \ \vee \ \text{is_bad_label } \tau \text{ then None} \\ &\text{else if } \text{chor_match } \tau \ (\text{Let } v \ p \ e \ c) \text{ then} \\ &\quad \text{case} \\ &\quad \quad \text{some } r. \ \exists \ cl. \\ &\quad \quad \quad \text{eval_exp } cl \ (\text{localise } s \ p) \ e = r \wedge r \neq \text{Timeout} \\ &\quad \text{of} \\ &\quad \quad \text{None} \Rightarrow \text{None} \\ &\quad \quad | \text{ Some } (\text{Value } ev) \Rightarrow \text{Some } (s[(v, p) := ev], c) \\ &\quad \quad | \text{ Some } \text{TypeError} \Rightarrow \text{None} \\ &\quad \quad | \text{ Some } (\text{Exn } v_6) \Rightarrow \text{Some } (s, 0) \\ &\quad \quad | \text{ Some } \text{Timeout} \Rightarrow \text{None} \\ &\text{else syncTrm } (k - 1) \ (\text{chor_tl } s \ (\text{Let } v \ p \ e \ c)) \ \tau \end{aligned}$$

Proposition 4 (Synchronous transitions up to a given label). *Given an initial state s , a choreography c , an integer k , and a label τ , we have*

$$\begin{aligned} \text{fv}(c) &\subseteq \text{domain}(s) \wedge \text{no_self_communication } c \wedge \\ \text{syncTrm } k \ (s, c) \ \tau &= \text{Some } (s', c') \Rightarrow \text{trans_sync } (s, c) \ (s', c') \end{aligned}$$

The function syncTrm takes a counter k , a state-choreography pair (s, c) , and a label τ . It advances the choreography synchronously until the label τ is consumed, returning the choreography after this consumption. If the choreography reaches its end or if the counter decreases to zero, it returns None instead.

We modify the cases for **If**, **Com**, and **Let** within syncTrm to align with the transition rules for the enriched Kalas (as defined in Definition 20). The function $\text{chor_match } \tau \ c$ checks whether τ matches c ; if it does, a synchronous transition from c will consume the label τ . The function $\text{chor_tl } s \ c$ takes a state and a choreography, returning the updated state-choreography pair after a synchronous transition from (s, c) .

Additionally, $\text{is_bad_label } \tau$ checks if τ is equal to $\text{let } v@p, \text{Timeout}$, which should not occur given the strong normalisation property of Sprinkles. Lastly, $\text{trans_sync } (s, c) \ (s', c')$ is defined as the reflective transitive closure on synchronous transitions (i.e., transitions where the list of asynchronous labels is empty; see also Definition 20). Rules for parallel transitions and definition for chor_tl can be found in Appendix A.

The proof for Proposition 4 follows from the definitions above. For the `Let` case, `is_bad_label` τ helps us filter out `let  $v@p$ , Timeout`, as by Definition 22

$$\text{syncTrm } k \ (s, \text{Let } v \ p \ e \ c) \ (\text{let } v@p, \text{Timeout})$$

will return `None`.

□

Concluding Remarks

7.1 Conclusion

In this thesis, we present a simply typed lambda calculus, **Sprinkles**, with functional big-step semantics and function closures. To restore the property for transitions in the larger state (Proposition 3) of Kalas, we adjust the evaluation rule in **Sprinkles**. Specifically, when evaluating a function expression, we return a restricted function closure.

We establish that **Sprinkles** enjoys both type soundness and strong normalisation. Our type soundness proof follows a standard approach. However, evaluating function expressions into restricted closures requires us to exhibit a minimal typing environment for a typed expression in the function case of the type soundness proof.

Function big-step semantics means that the evaluation function for **Sprinkles** is total and deterministic, making the strong normalisation proof equivalent to a weak one. Our proof approach is mostly standard, namely induction over types. Although the use of function big-step semantics and environments requires us to define strongly normalising values and environments consisting of strongly normalising values, which are not typically addressed in a standard proof outlined by [Abel et al. \(2019\)](#).

We demonstrate that **Sprinkles** can be integrated into Kalas, enabling more streamlined choreographic programming with ability to write concrete local computation codes. We also provide typing rules to Kalas enriched with **Sprinkles**. Our proof results indicate that the strong normalisation property of **Sprinkles** implies progress for the enriched Kalas. Additionally, the combination of strong normalisation and type soundness of **Sprinkles** implies type preservation for non-recursive, synchronous transitions in the enriched Kalas. This type preservation result differs from the one by [Hirsch and Garg \(2022\)](#), where the type preservation property of their assumed local language implies type preservation for the choreography.

7.2 Future Work

Immediate future work involves proving type preservation for recursive and asynchronous transitions in the enriched Kalas. Once type preservation is established for all transitions, we can assert

the termination property for a typed choreography: if we have $\Gamma, \Theta \vdash c \checkmark$, $\text{chorEnvtype } \Gamma s$, and $\text{chorEnvsn } \Gamma s$, then there exists a state-choreography pair (s', c') that is related to (s, c) through the reflexive transitive closure of the transition relation defined in Definition 20.

Another immediate task involves updating the scripts for later stages of Kalas's compilation, such as endpoint projection. These scripts may still rely on the original Kalas syntax, so modifications are needed to align them with the changes in syntax and transitions introduced by the enriched Kalas.

In the long term, future work could focus on introducing more message types in the enriched Kalas. Currently, messages are limited to strings, but we can envision processes communicating using richer data types, such as integers, booleans, or even function closures.

Another important area of future work is implementing a more robust exception-handling mechanism for the enriched Kalas. Currently, the choreography transitions directly to $\mathbf{0}$ in response to communication or local computation exceptions. Drawing inspiration from the work of [Giallorenzo et al. \(2024\)](#); [Montesi and Peressotti \(2017\)](#), one could introduce try-catch blocks to manage exceptions and implement recovery strategies. This approach would allow the choreography to handle errors more gracefully, avoiding an immediate transition to $\mathbf{0}$.

Sprinkles uses a functional big-step semantics and looks up values for variables in an environment. It would be interesting as well to implement a small-step semantics with a substitution model for **Sprinkles** to explore how this change might affect the progress and type preservation proofs for the enriched choreography.

Appendix

Well-founded lexicographic induction principle for Definition 8

$$\begin{aligned}
& (\forall c \ E \ str. \ P \ c \ E \ (\text{Var } str)) \wedge \\
& (\forall c \ E \ str. \ P \ c \ E \ (\text{StrLit } str)) \wedge \\
& (\forall c \ E \ n. \ P \ c \ E \ (\text{IntLit } n)) \wedge (\forall c \ E \ b. \ P \ c \ E \ (\text{BoolLit } b)) \wedge \\
& (\forall c \ E \ bop \ e_1 \ e_2. \\
& \quad P \ c \ E \ e_1 \wedge P \ c \ E \ e_2 \Rightarrow P \ c \ E \ (\text{BinOp } bop \ e_1 \ e_2)) \wedge \\
& (\forall c \ E \ uop \ e. \ P \ c \ E \ e \Rightarrow P \ c \ E \ (\text{Uop } uop \ e)) \wedge \\
& (\forall c \ E \ e_1 \ e_2 \ e_3. \\
& \quad (\forall b. \ b \Rightarrow P \ c \ E \ e_2) \wedge (\forall b. \neg b \Rightarrow P \ c \ E \ e_3) \wedge P \ c \ E \ e_1 \Rightarrow \\
& \quad P \ c \ E \ (\text{If } e_1 \ e_2 \ e_3)) \wedge \\
& (\forall c \ E \ s \ e_1 \ e_2. \\
& \quad (\forall v_1. \ P \ c \ E[s := v_1] \ e_2) \wedge P \ c \ E \ e_1 \Rightarrow P \ c \ E \ (\text{Let } s \ e_1 \ e_2)) \wedge \\
& (\forall c \ E \ s \ e. \ P \ c \ E \ (\text{Fn } s \ e)) \wedge \\
& (\forall c \ E \ e_1 \ e_2. \\
& \quad (\forall v_1 \ s \ e \ E_1 \ v_2. \\
& \quad \quad c > 0 \wedge v_1 = \text{Clos } s \ e \ E_1 \Rightarrow P \ (c - 1) \ E_1[s := v_2] \ e) \wedge \\
& \quad (c > 0 \Rightarrow P \ c \ E \ e_1) \wedge (c > 0 \Rightarrow P \ c \ E \ e_2) \Rightarrow \\
& \quad P \ c \ E \ (\text{App } e_1 \ e_2)) \wedge \\
& (\forall c \ E \ e \ s_1 \ e_1 \ s_2 \ e_2. \\
& \quad (\forall v \ v_0. \ v = \text{SumLV } v_0 \Rightarrow P \ c \ E[s_1 := v_0] \ e_1) \wedge \\
& \quad (\forall v \ v'_0. \ v = \text{SumRV } v'_0 \Rightarrow P \ c \ E[s_2 := v'_0] \ e_2) \wedge P \ c \ E \ e \Rightarrow \\
& \quad P \ c \ E \ (\text{Case } e \ s_1 \ e_1 \ s_2 \ e_2)) \Rightarrow \forall v \ v_1 \ v_2. \ P \ v \ v_1 \ v_2
\end{aligned}$$

Definition of eval_uop

```

eval_uop Not v  $\stackrel{\text{def}}{=}$  do b  $\leftarrow$  getB v; Value (BoolV ( $\neg b$ )) od
eval_uop NumOf v  $\stackrel{\text{def}}{=}$ 
do
  s  $\leftarrow$  getS v;
  case string_to_int2 s of
    None  $\Rightarrow$  Exn BadStr
  | Some i  $\Rightarrow$  Value (IntV i)
od
eval_uop StrOf v  $\stackrel{\text{def}}{=}$ 
do i  $\leftarrow$  getI v; Value (StrV (int_to_string i)) od
eval_uop Fst v  $\stackrel{\text{def}}{=}$  do v  $\leftarrow$  getP v; Value (fst v) od
eval_uop Snd v  $\stackrel{\text{def}}{=}$  do v  $\leftarrow$  getP v; Value (snd v) od
eval_uop SumL v  $\stackrel{\text{def}}{=}$  Value (SumLV v)
eval_uop SumR v  $\stackrel{\text{def}}{=}$  Value (SumRV v)

```

Definition of eval_bop

```

eval_bop Add v1 v2  $\stackrel{\text{def}}{=}$  binlop (+) v1 v2
eval_bop Concat v1 v2  $\stackrel{\text{def}}{=}$  binSop STRCAT v1 v2
eval_bop Mult v1 v2  $\stackrel{\text{def}}{=}$  binlop ( $\times$ ) v1 v2
eval_bop Div v1 v2  $\stackrel{\text{def}}{=}$ 
  if getI v2  $\neq$  Value 0 then binlop (div) v1 v2 else Exn Div0
eval_bop Mod v1 v2  $\stackrel{\text{def}}{=}$ 
  if getI v2  $\neq$  Value 0 then binlop (MOD) v1 v2 else Exn Div0
eval_bop Less v1 v2  $\stackrel{\text{def}}{=}$ 
  do i1  $\leftarrow$  getI v1; i2  $\leftarrow$  getI v2; Value (BoolV (i1 < i2)) od
eval_bop And v1 v2  $\stackrel{\text{def}}{=}$  binBop ( $\wedge$ ) v1 v2
eval_bop Or v1 v2  $\stackrel{\text{def}}{=}$  binBop ( $\vee$ ) v1 v2
eval_bop Eq v1 v2  $\stackrel{\text{def}}{=}$  Value (BoolV (v1 = v2))
eval_bop Sub v1 v2  $\stackrel{\text{def}}{=}$  binlop ( $-$ ) v1 v2
eval_bop Pair v1 v2  $\stackrel{\text{def}}{=}$  Value (PairV v1 v2)

```

Parallel transitions for Kalas enriched with Sprinkles

$$\begin{array}{c}
\text{CS-IFSWAP} \frac{s \triangleright c_1 \xrightarrow[l]{\alpha} s' \triangleright c'_1 \quad s \triangleright c_2 \xrightarrow[l']{\alpha} s' \triangleright c'_2 \quad l \simeq l' \quad p \notin \text{fp}(\alpha)}{s \triangleright \text{if } v@p \text{ then } c_1 \text{ else } c_2 \xrightarrow[l]{\alpha} s' \triangleright \text{if } v@p \text{ then } c'_1 \text{ else } c'_2} \\
\\
\text{CS-COMSWAP} \frac{s \triangleright c \xrightarrow[l]{\alpha} s' \triangleright c' \quad p_1 \notin \text{fp}(\alpha) \quad p_2 \notin \text{fp}(\alpha)}{s \triangleright p_1.v_1 \Rightarrow p_2.v_2; c \xrightarrow[l]{\alpha} s' \triangleright p_1.v_1 \Rightarrow p_2.v_2; c'} \\
\\
\text{CS-SELSWAP} \frac{s \triangleright c \xrightarrow[l]{\alpha} s' \triangleright c' \quad p_1 \notin \text{fp}(\alpha) \quad p_2 \notin \text{fp}(\alpha)}{s \triangleright p_1 \Rightarrow p_2[b]; c \xrightarrow[l]{\alpha} s' \triangleright p_1 \Rightarrow p_2[b]; c'} \\
\\
\text{CS-LETSWAP} \frac{s \triangleright c \xrightarrow[l]{\alpha} s' \triangleright c' \quad p \notin \text{fp}(\alpha)}{s \triangleright \text{Let } v \text{ p e } c \xrightarrow[l]{\alpha} s' \triangleright \text{Let } v \text{ p e } c'}
\end{array}$$

Definition of `chor_tl`

$$\begin{aligned}
\text{chor_tl } s \mathbf{0} &\stackrel{\text{def}}{=} (s, \mathbf{0}) \\
\text{chor_tl } s \ v &\stackrel{\text{def}}{=} (s, v) \\
\text{chor_tl } s \ (\mu \text{dn}. c) &\stackrel{\text{def}}{=} (s, c[\text{dn} := \mu \text{dn}. c]) \\
\text{chor_tl } s \ (p.v \Rightarrow q.x; c) &\stackrel{\text{def}}{=} \\
&\text{case } s \ (v, p) \text{ of} \\
&\quad \text{None} \Rightarrow (s, \mathbf{0}) \\
&\quad | \text{Some } (\text{IntV } v_{12}) \Rightarrow (s, \mathbf{0}) \\
&\quad | \text{Some } (\text{StrV } d) \Rightarrow (s[(x, q) := \text{StrV } d], c) \\
&\quad | \text{Some } (\text{BoolV } v_{14}) \Rightarrow (s, \mathbf{0}) \\
&\quad | \text{Some } (\text{PairV } v_{15} \ v_{16}) \Rightarrow (s, \mathbf{0}) \\
&\quad | \text{Some } (\text{SumLV } v_{17}) \Rightarrow (s, \mathbf{0}) \\
&\quad | \text{Some } (\text{SumRV } v_{18}) \Rightarrow (s, \mathbf{0}) \\
&\quad | \text{Some } (\text{Clos } v_{19} \ v_{20} \ v_{21}) \Rightarrow (s, \mathbf{0}) \\
\text{chor_tl } s \ (p \Rightarrow q[b]; c) &\stackrel{\text{def}}{=} (s, c) \\
\text{chor_tl } s \ (\text{Let } v \text{ p e } c) &\stackrel{\text{def}}{=} \\
&\text{case} \\
&\quad \text{some } ev. \exists cl. \text{eval_exp } cl \ (\text{localise } s \ p) \ e = \text{Value } ev \\
&\text{of} \\
&\quad \text{None} \Rightarrow (s, \mathbf{0}) \\
&\quad | \text{Some } ev \Rightarrow (s[(v, p) := ev], c) \\
\text{chor_tl } s \ (\text{if } v@p \text{ then } c_1 \text{ else } c_2) &\stackrel{\text{def}}{=} \\
&\text{if } s \ (v, p) = \text{BoolV } \mathbf{T} \text{ then } (s, c_1) \\
&\text{else if } s \ (v, p) = \text{BoolV } \mathbf{F} \text{ then } (s, c_2) \\
&\text{else } (s, \mathbf{0})
\end{aligned}$$

Bibliography

- ABEL, A.; ALLAIS, G.; HAMEER, A.; PIENKA, B.; MOMIGLIANO, A.; SCHÄFER, S.; AND STARK, K., 2019. Poplmark reloaded: Mechanizing proofs by logical relations. *Journal of Functional Programming*, 29 (2019), e19. [Cited on pages 11 and 47.]
- CARBONE, M.; HONDA, K.; AND YOSHIDA, N., 2007. Structured communication-centred programming for web services. In *Programming Languages and Systems: 16th European Symposium on Programming, ESOP 2007, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2007, Braga, Portugal, March 24-April 1, 2007. Proceedings 16*, 2–17. Springer. [Cited on page 5.]
- CARBONE, M. AND MONTESI, F., 2013. Deadlock-freedom-by-design: multiparty asynchronous global programming. 48, 1 (2013), 263–274. doi:10.1145/2480359.2429101. <https://dl.acm.org/doi/10.1145/2480359.2429101>. [Cited on pages 1 and 9.]
- CRUZ-FILIPPE, L.; GRAVERSEN, E.; LUGOVIĆ, L.; MONTESI, F.; AND PERESSOTTI, M., 2022. Functional choreographic programming. In *International Colloquium on Theoretical Aspects of Computing*, 212–237. Springer. [Cited on page 10.]
- CRUZ-FILIPPE, L. AND MONTESI, F., 2017. A core model for choreographic programming. In *Formal Aspects of Component Software: 13th International Conference, FACS 2016, Besançon, France, October 19-21, 2016, Revised Selected Papers 13*, 17–35. Springer. [Cited on pages 1, 2, 6, and 9.]
- GIALLORENZO, S.; MONTESI, F.; AND PERESSOTTI, M., 2024. Choral: Object-oriented choreographic programming. *ACM Transactions on Programming Languages and Systems*, 46, 1 (2024), 1–59. [Cited on pages 10 and 48.]
- GORDON, M. J., 1988. Hol: A proof generating system for higher-order logic. In *VLSI specification, verification and synthesis*, 73–128. Springer. [Cited on page 6.]
- HALLAL, R.; JABER, M.; AND ABDALLAH, R., 2018. From global choreography to efficient distributed implementation. In *2018 International Conference on High Performance Computing & Simulation (HPCS)*, 756–763. doi:10.1109/HPCS.2018.00122. https://ieeexplore.ieee.org/abstract/document/8514427?casa_token=B9uMnW0mxFEAAAAA:DMmhwgQZJnHAX6o6p-EHBs4K9rct4pEKen9fdt2CXHC6NOWZxpHT5FSZZFAchGBDjiqmPeviH1U. [Cited on page 1.]
- HARDIN, T.; JAUME, M.; PESSAUX, F.; AND DONZEAU-GOUGE, V. V., 2021. *Concepts and semantics of programming languages 1: a semantical approach with OCaml and Python*. John Wiley & Sons. [Cited on page 19.]

- HIRSCH, A. K. AND GARG, D., 2022. Pirouette: higher-order typed functional choreographies. 6 (2022), 23:1–23:27. doi:10.1145/3498684. <https://dl.acm.org/doi/10.1145/3498684>. [Cited on pages 1, 9, and 47.]
- KUMAR, R.; MYREEN, M. O.; NORRISH, M.; AND OWENS, S., 2014. Cakeml: a verified implementation of ml. *ACM SIGPLAN Notices*, 49, 1 (2014), 179–191. [Cited on page 6.]
- MONTESI, F. AND PERESSOTTI, M., 2017. Choreographies meet communication failures. *arXiv preprint arXiv:1712.05465*, (2017). [Cited on pages 10 and 48.]
- MONTESI, F. AND YOSHIDA, N., 2013. Compositional choreographies. In *CONCUR 2013 – Concurrency Theory* (Berlin, Heidelberg, 2013), 425–439. Springer. doi:10.1007/978-3-642-40184-8_30. [Cited on page 1.]
- MULLENDER, S., 1990. *Distributed systems*. ACM. [Cited on page 5.]
- NEEDHAM, R. M. AND SCHROEDER, M. D., 1978. Using encryption for authentication in large networks of computers. 21, 12 (1978), 993–999. doi:10.1145/359657.359659. <https://dl.acm.org/doi/10.1145/359657.359659>. [Cited on page 1.]
- OWENS, S. AND LAURENT, T., 2016. Prove that simply typed call-by-value lambda calculus programs always terminate. <https://github.com/HOL-Theorem-Prover/HOL/tree/devel/op/examples/fun-op-sem/cbv-lc>. Accessed: 2024-10-19. [Cited on page 11.]
- OWENS, S.; MYREEN, M. O.; KUMAR, R.; AND TAN, Y. K., 2016. Functional big-step semantics. In *Programming Languages and Systems: 25th European Symposium on Programming, ESOP 2016, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2016, Eindhoven, The Netherlands, April 2–8, 2016, Proceedings 25*, 589–615. Springer. [Cited on pages 2, 11, and 13.]
- PIERCE, B. C., 2002. *Types and programming languages*. MIT press. [Cited on pages 21 and 22.]
- POHJOLA, J. Å.; GÓMEZ-LONDOÑO, A.; SHAKER, J.; AND NORRISH, M., 2022. Kalas: A verified, end-to-end compiler for a choreographic language. In *13th International Conference on Interactive Theorem Proving (ITP 2022)*. Schloss-Dagstuhl-Leibniz Zentrum für Informatik. [Cited on pages 1 and 9.]
- SLIND, K. AND NORRISH, M., 2008. A brief overview of hol4. In *International Conference on Theorem Proving in Higher Order Logics*, 28–32. Springer. [Cited on pages 2 and 6.]
- W3C WS-CDL WORKING GROUP, 2005. Web Services Choreography Description Language Version 1.0. Technical report, World Wide Web Consortium (W3C). <http://www.w3.org/TR/2004/WD-ws-cdl-10-20040427/>. Accessed: October 14, 2024. [Cited on page 5.]